



Departamentul Automatică și Informatică Industrială
Facultatea Automatică și Calculatoare
Universitatea POLITEHNICA din București



Echipamente pentru conducerea proceselor

- I. [SISTEME DE ACHIZIȚIE DE DATE SI COMUNICATII](#)
- II. [AUTOMATE PROGRAMABILE](#)

Bibliografie admitere master, an universitar 2020-2021, pentru
[programele de master coordonate de Departamentul AII:](#)

Automatica si Informatica Industriala (AII)
Managementul si Protectia Informatiei (MPI)
Robotics and Automation (RA)
Prelucrari Complexe de Semnal in Aplicatii Multimedia (PCSAM)
Sisteme Informatice in Medicina (SIM)
Service Engineering and Management (SEM)



I. SISTEME DE ACHIZIȚIE DE DATE ȘI COMUNICAȚII

1. Sisteme de achiziție de date (SAD)

1.1. Modalități de achiziție a datelor de proces

Un sistem de reglare automată a proceselor impune măsurarea parametrilor din lumea reală, transformarea acestora în semnale electrice sau blocuri de date compatibile cu sistemul și memorarea lor în vederea utilizării la elaborarea legii de comandă a procesului. De fapt, această acțiune constituie achiziția datelor de proces. Transformarea mărimilor măsurate din lumea reală în semnale de natură electrică este realizată de către sistemul de senzori. Sistemul care are rolul de a prelua semnalele analogice, numerice sau logice, obținute de la sistemul de senzori, și de a le converti în semnale numerice standardizate, în scopul memorării, transmiterii, prelucrării și vizualizării informației achiziționate, se numește *sistem de achiziție a datelor* (SAD).

În aplicațiile industriale sunt necesare achiziția simultană și prelucrarea corelată, în timp real, pentru mai mulți parametri de proces. În majoritatea cazurilor, pe de o parte, acești parametri au o variație continuă, iar pe de altă parte, echipamentele de prelucrare (calcul) sunt numerice, ceea ce impune conversia analog-numerică pentru datele culese. După prelucrarea semnalelor achiziționate din proces, comenzile și deciziile corespunzătoare, emise de către sistemul de comandă, sunt trimise spre elementele de execuție, pentru finalizarea conducerii procesului. Dacă elementele de execuție sunt analogice, este necesară conversia numericanalogică a semnalelor obținute ca rezultat al prelucrării. Deoarece, de regulă, există mai multe elemente de execuție comandate de către sistemul de calcul, distribuția comenzilor se efectuează de către un sistem complementar sistemului de achiziție a datelor, numit *sistem de distribuție a datelor* (SDD). Din cele precizate anterior, se poate considera că funcțiile principale ale unui SAD sunt următoarele: adaptarea semnalelor preluate de la senzori, eșantionarea semnalelor adaptate, multiplexarea canalelor (semnalelor), conversia analognumerică, memorarea datelor obținute prin conversie și comunicarea datelor numerice la sistemul de prelucrare. Analog, se pot deduce funcțiile principale ale unui SDD: demultiplexarea datelor și memorarea acestora pentru comanda elementelor de execuție, conversia numeric-analogică și adaptarea în putere a semnalelor de ieșire.

În prezent este din ce în ce mai mult folosit termenul SCADA (*Supervisory Control And Data Acquisition*), care se referă la un sistem distribuit de măsurare și control utilizat, de obicei, la monitorizarea și controlul unor procese complexe fizice, chimice sau de transport. Aceste sisteme conțin atât resurse hardware cât și resurse software, combinate eficient. Un sistem SCADA (fig.16.1) are trei componente principale: setul terminalelor $T_1, \dots, T_n$, care sunt controlere programabile ($CP_1, \dots, CP_n$), infrastructura de comunicații (IC) și stația principală de control (SPC), ce conține interfața om-mașină (IOM). Funcțiile de achiziție de date și cea mai mare parte a funcțiilor de comandă sunt realizate la nivelul controlerelor programabile. Funcția de bază a stației principale de control este cea de supervizare. $SP_1, \dots, SP_n$ reprezintă subprocesele ale procesului condus. În cazul cel mai simplu, SPC este un PC obișnuit.

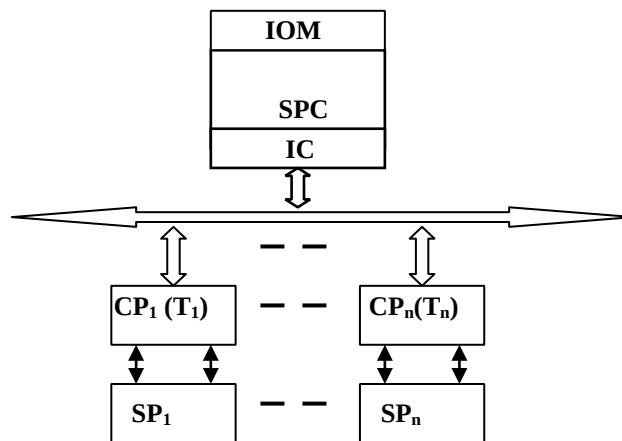


Fig.16.1. Structura unui sistem SCADA.

Datorită dezvoltării tehnologiei informației, se remarcă o utilizare din ce în ce mai largă a terminalelor inteligente, capabile să efectueze anumite sarcini privind conducerea locală, fără intervenția stației centrale. Astfel, o parte din software-ul proiectat pentru SCADA este rulat pe controlerele programabile.

Asocierea sistemelor de achiziție a datelor cu circuite specifice tehnicii de calcul a condus la apariția unor echipamente deosebit de performante destinate achiziției și prelucrării datelor de proces. În prezent, există trei soluții la problema introducerii tehnicii de calcul în procesul de achiziție a datelor de la sistemul de senzori, și anume:

- realizarea sistemului de achiziție sub forma unui echipament de sine stătător; □ încorporarea unor sisteme cu microprocesor la nivelul blocului de senzori și conectarea senzorilor inteligenți pe o magistrală comună legată la calculatorul principal;
- introducerea unei plăci de achiziție a datelor în interiorul calculatorului principal. Principalele avantaje ale utilizării tehnicii de calcul la realizarea acestor echipamente sunt următoarele:
 - creșterea flexibilității (prin programare);
 - reducerea numărului de componente utilizate;
 - reducerea dimensiunilor echipamentelor;
 - reducerea consumului energetic;
 - mărirea preciziei de măsurare (datorită autocalibrării, reducerii zgomotului prin filtrare numerică, corectării automate a diverselor erori sistematice);
 - automatizarea echipamentului (prin preluarea funcțiilor de comandă de către subsistemul cu microprocesor);
 - integrarea simplă a sistemelor de măsură și conducere a proceselor.

În prezent, au început să fie utilizate pe scară tot mai largă *microcontroler-ele* pe 8 și 16 biți, care compensează o parte din dezavantajele microprocesoarelor tradiționale. Arhitectura unui microcontroler a fost astfel proiectată încât să ofere o versatilitate superioară celei a microprocesoarelor tradiționale, datorită următoarelor aspecte:

- organizarea internă sub forma unui număr de trei, patru sau cinci porturi bidirecționale, a căror funcționalitate poate fi stabilită de utilizator;
- extinderea spațiului de adresare prin separarea, din punctul de vedere al accesului, a memoriei de date față de memoria de program;
- înglobarea, în structura microcontrolerului, a unei memorii de program de tip

EPROM de capacitate relativ mică, ce poate fi parțial sau integral accesibilă operatorului pentru programul de aplicații;

- înglobarea, în structura internă, a unor circuite auxiliare, cum ar fi: circuite de interfațare (interfețe seriale programabile de comunicație, interfețe paralele programabile bidirecționale), circuite de numărare programabile, convertoare analog-digitale, convertoare digital-analogice etc.;

- operarea la o frecvență a ceasului de câteva ori mai mare decât în cazul unui microprocesor tradițional, ceea ce permite creșterea vitezei de prelucrare, aproximativ în aceeași măsură;

- îmbunătățirea substanțială a setului de instrucțiuni al microcontrolerului, păstrând compatibilitatea cu cele ale microprocesoarelor tradiționale;

- scăderea substanțială a puterii consumate de la sursa de alimentare, creșterea fiabilității și a siguranței în exploatare, datorită tehnologiei de realizare CMOS, folosită pentru implementarea majorității microcontrolerelor actuale.

În afară de microprocesor, care este componenta cea mai complexă, ale cărei caracteristici determină în esență structura și performanțele unității centrale, se disting și alte elemente de bază ale echipamentelor cu tehnică de calcul, cum ar fi: memoriile nevolatile, memoriile volatile, circuitele de interfațare și decodificatoare.

Există aplicații, în zone aflate la distanță, unde nu este posibilă, sau nu se dorește o cablare permanentă, de tipul magistrelor de câmp, la sistemul central de achiziție și prelucrare a datelor: studii de câmp, monitorizarea transporturilor, studii asupra calității mediului, cercetări generale, achiziții și înregistrări de date din diverse domenii industriale (industria petrolului, materialelor de construcții, energiei electrice, metalurgică, chimică etc.). În aceste aplicații se folosesc echipamente digitale speciale, înrudite cu sistemele de achiziție a datelor numite *data logger*-e (DL). Acestea sunt instrumente electronice digitale, dotate, uneori, cu tehnică de calcul de nivel înalt, care înregistrează (memorează), pe o perioadă de timp, date provenind din măsurarea unor semnale analogice, numerice, frecvențiale, evenimente sau chiar mici programe de operare. La un data logger pot fi conectați direct mai mulți senzori. În ceea ce privește relația dintre sistemul de achiziție a datelor și data logger-ul, se poate spune că un data logger este un sistem de achiziție a datelor, pe când un sistem de achiziție nu este în mod necesar un data logger.

De cele mai multe ori, DL este portabil, sub forma unui echipament de sine stătător, de mici dimensiuni, alimentat cu baterii și echipat cu microprocesor (microcontroler), memorie de date (uneori, de mari dimensiuni, pentru a acoperi perioade lungi, de zile, săptămâni, luni), memorie de program și interfețe I/O. El utilizează un software specific pentru următoarele activități de bază: inițializare, achiziție, filtrare semnal, memorare și transfer de date pe un PC [1].

Cerințele care se impun unui DL portabil sunt următoarele:

- achiziționarea datelor în timp real, stocarea și apoi descărcarea lor într-un PC pentru prelucrări ulterioare (filtrări complexe, compuneri de semnale, calcule de armonici, corelări de semnale, stabilirea unor legi de reglare etc.);

- utilizarea în cadrul unor procese relativ lente, deoarece DL operează de obicei la viteze de eșantionare mici (mai mici de 5 eșantioane/secundă);

- multifuncționalitatea, cu aplicații în monitorizarea, conducerea, diagnosticarea și identificarea proceselor;

- flexibilitatea structurală, care să permită alegerea configurației optime din punctul de vedere preț/performanță, ținând seama de o serie de elemente, cum ar fi: numărul de intrări analogice, numărul și tipul elementelor de adaptare, tipul și puterea procesorului, tipul și mărimea memoriei, afișajul, tastatura, accesoriile;
- suportul hardware să posede un mediu de dezvoltare de nivel înalt, de tip calculator personal, dată fiind importanța posibilităților de dezvoltare și modificare software;
- satisfacerea unei game largi de aplicații, prin facilitățile oferite privind: numărul de înregistrări, precizia de măsurare, durata de independență energetică, informațiile de caracterizare a măsurărilor.

În principiu, funcționarea DL constă în următoarele etape:

- la început, DL este conectat la PC. Aici, prin programare, este inițializat, selectând parametrii de achiziție: intervalele de timp, domeniile de măsurare, momentul de start, numărul de eșantioane memorate etc.;
- apoi, DL este deconectat de la PC și este plasat la locația dorită, în câmp, unde înregistrează, corespunzător programării, fiecare măsurare și o memorează, împreună cu informațiile despre timp;
- după aceasta, DL este deconectat de la proces (de la senzori), este transportat la PC, unde datele sunt descărcate, iar DL este reinițializat și procesul se reia.

Până acum au fost analizate metodele de achiziție a datelor de la senzori individuali sau de la grupări relativ mici de senzori. În cazul matricelor de senzori, considerate grupări mari, dense (senzori vizuali și tactili), modalitățile de achiziție a datelor de la acestea sunt particulare și necesită integrarea, alături de gruparea de senzori propriu-zisă, a unor registre de deplasare care fac posibilă citirea serială din matricea de senzori. De exemplu, un senzor de imagine CCD liniar (fig.16.2) conține, alături de bareta de fotoelemente, două registre de deplasare cuplate prin sarcină care serializează blocul de date (vectorul) în vederea citirii la ieșire.

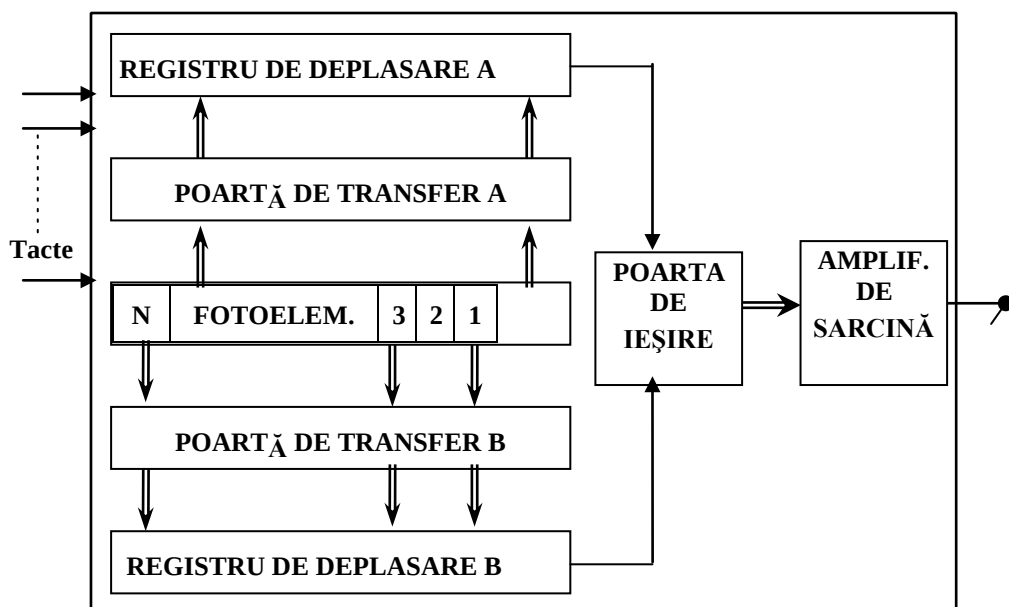


Fig. 16.2. Achiziția datelor de la un senzor vizual liniar prin serializarea datelor.

1.2. Arhitecturi de sisteme de achiziție a datelor

În general, structura unui SAD conține următoarele blocuri funcționale de bază: circuite de adaptare pentru semnalele analogice de intrare, circuite de eșantionare și reținere, convertoare analog-numerice, multiplexoare (analogice sau numerice). După modul de amplasare a acestor elemente se deosebesc mai multe tipuri de arhitecturi de SAD, ce vor fi prezentate în continuare.

SAD cu multiplexarea semnalelor analogice de intrare. Multiplexarea semnalelor analogice de intrare reprezintă cea mai ieftină soluție de realizare a unui SAD deoarece necesită doar un singur element de eșantionare și reținere și un singur convertor analognumeric. Schema unui SAD cu multiplexarea semnalelor analogice de intrare este dată în figura 16.3.

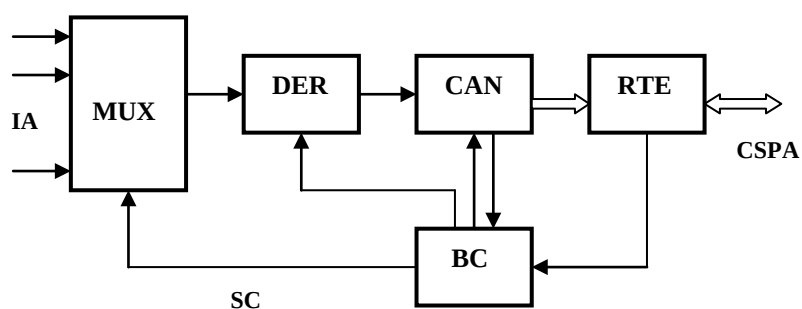


Fig. 16.3. Arhitectură SAD cu multiplexarea (analogică) intrărilor.

Semnificația notațiilor din figură este următoarea: IA – intrări (canale) analogice, MUX – multiplexor analogic, DER – dispozitiv de eșantionare și reținere, CAN – convertor analog-numeric, RTE – registru tampon de ieșire, BC – bloc de comandă, SC – semnal de selectare a canalului, CSPA – comunicație cu sistemul de prelucrare numerică și afișare. Semnalele de intrare, corespunzătoare mărimilor măsurate, trebuie să fie în gama cerută de CAN, de regulă $0 \div 10 V_{cc}$, sau alt semnal standardizat. Din această cauză, traductoarele, care furnizează semnalele IA, sunt prevăzute cu circuite de ieșire adecvate. MUX asigură, prin intermediul semnalului de control (SC), selectarea unui singur canal (semnal de intrare), pe durata unei perioade de eșantionare. DER menține constantă valoarea de tensiune aflată la ieșirea MUX, pe durata conversiei analog-numerice. Rezultatul conversiei, sub forma unui cuvânt binar de lungime: 8, 10, 12 sau 16 biți, este memorat temporar în RTE, până la preluarea lui de către sistemul de prelucrare. Blocul de comandă îndeplinește mai multe funcții, cele mai importante fiind următoarele:

- asigurarea baleierii canalelor de intrare, după o anumită lege, și selectarea canalului corespunzător în momentul de eșantionare;
- stabilirea frecvenței de eșantionare și a momentelor de eșantionare;
- startul conversiei analog-numerice;
- sesizarea sfârșitului de conversie și comanda memorării rezultatului de către RTE;
- asigurarea comunicației cu sistemul numeric de prelucrare.

Din această structură, elementul care introduce cea mai mare întârziere este, de regulă, CAN. Deoarece pentru fiecare canal este necesară conversia analog-numerică a semnalului respectiv, dacă numărul de canale este mare, frecvența de baleiere a canalelor va fi relativ mică, ceea ce face ca acest SAD să fie utilizat pentru procese lente.

SAD cu multiplexarea semnalelor eșantionate. Față de structura precedentă, structura de SAD din figura 16.4. necesită mai multe dispozitive de eșantionare și reținere (câte unul pentru fiecare canal de intrare): $DER_1, DER_2, \dots, DER_n$. În acest fel, eșantionarea semnalelor de pe canalele de intrare se face sincron (eșantioanele sunt prelevate din intrările sistemului la aceleași momente de timp – momentele de eșantionare). Viteza de achiziție este ceva mai mare decât în cazul multiplexării analogice a intrărilor. În rest, considerentele menționate rămân valabile și aici.

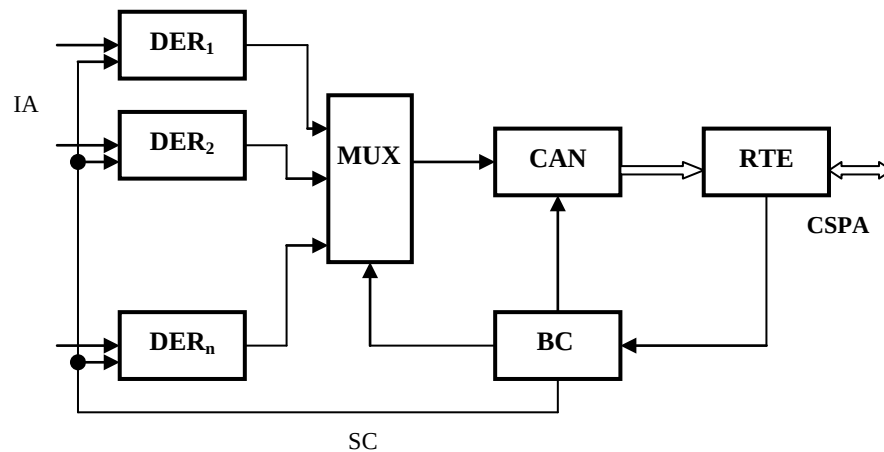


Fig.16.4. Arhitectura unui SAD cu multiplexarea semnalelor eșantionate.

SAD cu multiplexarea semnalelor numerice. SAD cu CAN pe fiecare canal și cu multiplexarea ieșirilor de la convertoarele analog-numerice reprezintă varianta cu prețul cel mai ridicat, dar și cea mai performantă. Eșantionarea și cuantizarea se fac sincronizat, pentru toate canalele.

Datorită faptului că fiecare canal are un convertor analog-numeric propriu, iar timpul de achiziție depinde, în principal, de timpul de conversie, acest tip de SAD are viteza de achiziție cea mai mare:

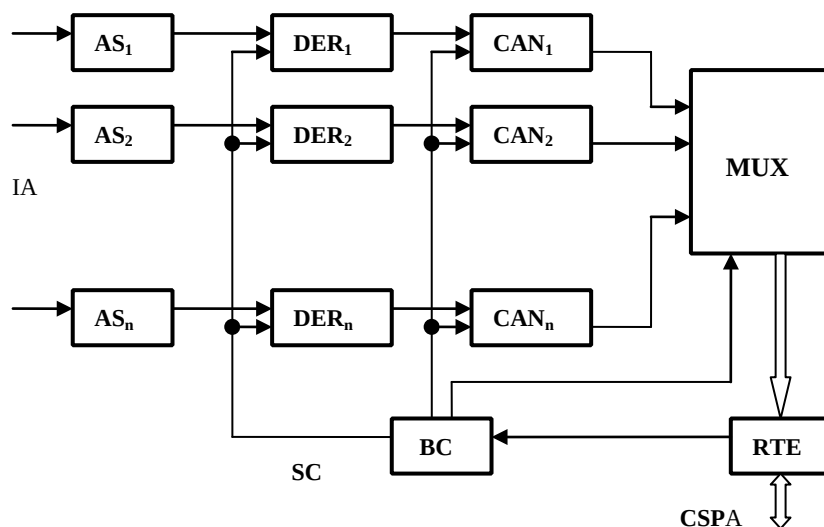


Fig. 16.5. Arhitectură SAD cu multiplexarea semnalelor numerice.

Structura unui SAD cu multiplexarea semnalelor numerice este prezentată în figura 16.5. La această schemă s-au pus în evidență adaptoarele de semnal de pe fiecare canal ($AS_1, AS_2, \dots, AS_n$), necesare pentru aducerea acestor semnale în gama de valori cerute de dispozitivele de eșantionare și reținere. Spre deosebire de schemele anterioare, multiplexorul MUX este numeric.

Structuri de sisteme de achiziție a datelor cu microprocesoare. Structura unui sistem de achiziție cu microprocesor este prezentată în figura 16.6. Semnificația notațiilor din figură este următoarea: ES-elemente sensibile, MM-mărimi de măsurat, CC-circuite de condiționare, CAN-convertor analog numeric, P(C) - microprocesor (microcontroler), DAdispozitiv de afișare, T-tastatură, I-interfață, DI/O-date de intrare/ieșire, P-programe.

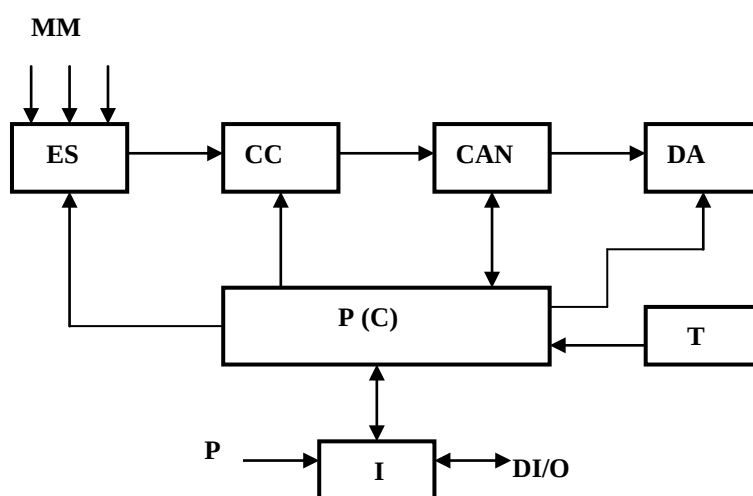


Fig. 16.6. Structura unui sistem de achiziție cu microprocesor.

În cadrul acestei structuri, microprocesorul îndeplinește următoarele funcții principale:

- crearea (prin program) a instrumentului de măsură dorit;
- controlul componentelor din structura echipamentului;
- asigurarea calibrării și stabilirea parametrilor funcționali ai echipamentului (de exemplu: viteza de eșantionare, modul de multiplexare, gama de intrare, amplificarea);
- corecția erorilor introduse de diverse influențe externe (zgomot) și interne (neliniarități, erori de nul etc);
- testarea blocurilor componente pentru detectarea eventualelor defecțiuni;
- prelucrarea numerică a rezultatelor măsurării.

În cazul în care tehnica de calcul este încorporată, sistemul de senzori programabil are structura din figura 16.7, semnificația notațiilor fiind următoarea: S-sistem de senzori, C-calculator, I-interfață, MS-magistrală standard.

Din punctul de vedere al numărului de procesoare, se disting structuri uniprocessor și structuri multiprocessor.

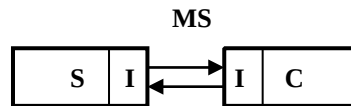


Fig. 16.7. Schema funcțională a unui sistem de senzori cu tehnică de calcul (încorporată).

În cazul în care tehnica de calcul este asociată, sistemul de achiziție este realizat pe o placă de circuit imprimat, care se introduce într-unul din locurile pentru extensii ale calculatorului, legătura fiind directă, pe magistrala internă a calculatorului. Ea se numește placă de achiziție de date de uz general și are structura prezentată în figura 16.8.

Semnificația notațiilor din figură este următoarea: M-multiplexor, AP-amplificator programabil, CAN-convertor analog-numeric, ML-memorie locală, IM-interfață cu magistrala, M/PC- magistrala calculatorului, BC-bloc de comandă, M_1 , M_2 -magistrale interne ale plăcii, I/O-intrări/ieșiri, ON- ieșiri numerice, IN-intrări numerice, CNA- convertor numeric-analogic, CNT-circuite de numărare și temporizare, I/A-interfață intrări analogice, I/ON-interfață intrări/ieșiri numerice.

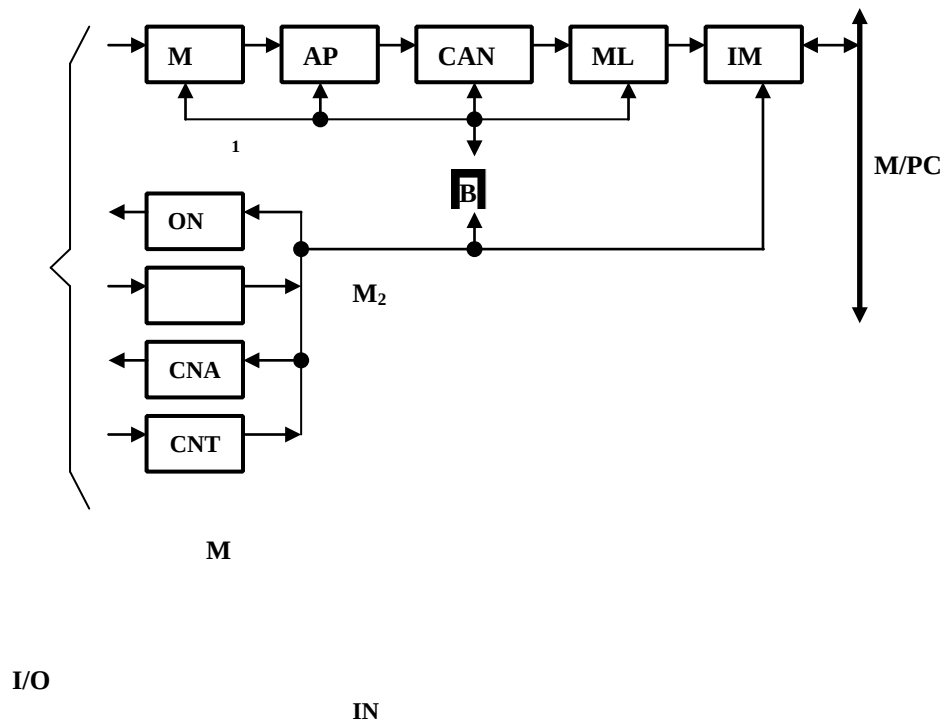


Fig. 16. 8. Structura generală a unei plăci de achiziție de date pentru calculator.

Structura unui data logger. Schema funcțională a unui DL portabil, pentru parametri de proces, este dată în figura 16.9. Semnificația notațiilor din figură este următoarea: IA – intrări analogice, IN – intrări numerice, ON – ieșiri numerice, CPU-I – unitate centrală de prelucrare de tip industrial, DI – display industrial de tip LCD, TI – tastatură de tip industrial,

IC – interfață pentru comunicație, IIA – interfață pentru intrări analogice, IION – interfață pentru intrări și ieșiri numerice, CCG – comunicație cu calculatorul gazdă.

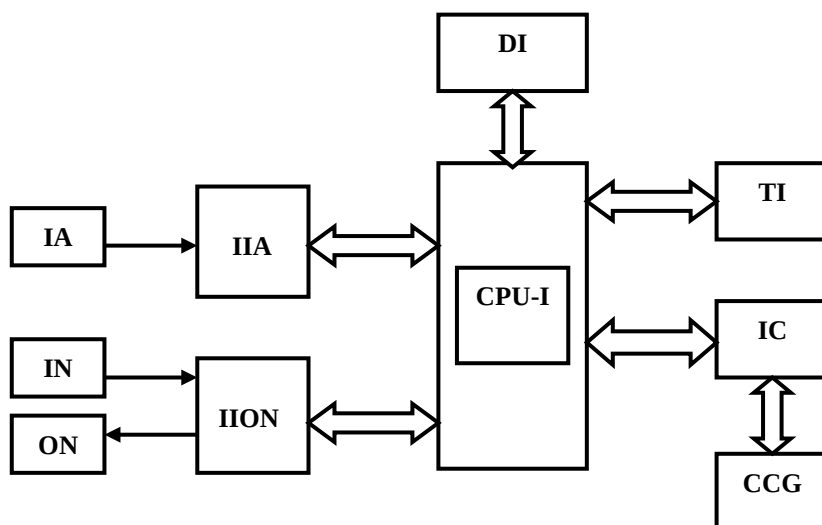


Fig.16.9. Schema funcțională a DL.

Software-ul pentru aplicații pe *DL* este alcătuit din două componente: □ software-ul pentru controller: *data logger*; □ software-ul pentru PC: *configuratorul*.

Data logger-ul constituie software-ul de execuție al aplicației. Acesta îndeplinește, în principal, următoarele funcții:

- achiziție de date simplă, cu posibilitatea încărcării datelor achiziționate pe un PC;
- achiziție de date cu supravegherea limitelor prestabilite; la depășirea unei limite, se poate activa una dintre ieșirile digitale specificate în configurator;
- achiziție de date sincronizate – cu indicarea unei referințe în timp;
- reglarea / sincronizarea unui proces, conform strategiei introduse în configurator.

Este important de remarcat faptul că, *data logger*-ul nu poate funcționa fără o configurație de rulare, specificată printr-o strategie. Aceasta este produsă, cu ajutorul *configuratorului*. În principiu, programul care rulează configurația este un executabil de timp real, capabil de a rula blocurile funcționale specificate de strategie, precum și de a face legăturile informaționale pentru acestea.

16.1.3. Principalele funcții și blocuri ale sistemelor de achiziție a datelor

În cadrul sistemelor de achiziție a datelor intervin trei operații caracteristice conversiei analog-numerice: eșantionarea, cuantizarea semnalului eșantionat și codificarea într-un cod binar a semnalului cuantizat. De cele mai multe ori, are loc și multiplexarea canalelor de intrare. Primele două operații sunt definitorii pentru conversia analog-numerică a datelor.

Eșantionarea și reținerea. Eșantionarea este operația prin care un semnal analogic $x(t)$, cu variație continuă în timp și în nivel, este transformat într-un semnal discret în timp, numit

semnal eșantionat. Referitor la reprezentarea semnalelor cu bandă limitată, prin secvențe de eșantioane, o importanță deosebită o are *teorema eșantionării*. În esență, teorema eșantionării afirmă faptul că, dacă un semnal x are bandă limitată, cu lărgimea de bandă $B[\text{Hz}]$, atunci el poate fi determinat, în mod unic, prin eșantioane luate cu frecvența f_e [eșantioane/secundă], unde $f_e \geq 2B$. Astfel, se stabilește limita inferioară a frecvenței de eșantionare: $f_{lim} = 2B$. Dacă frecvența de eșantionare f_e este mult mai mare decât frecvența limită, atunci se spune că se efectuează o supraeșantionare.

În cazul (cel mai frecvent) în care momentele de eșantionare sunt echidistante, atunci eșantionarea se numește *uniformă*.

Pentru a putea folosi valorile eșantionate în cadrul procesului de achiziție este necesar ca acestea să rămână constante pe toată durata de prelucrare ulterioară (cuantizare, codificare). Astfel, dispozitivul de eșantionare este combinat cu o memorie analogică de scurtă durată (depinzând, în principal, de timpul de conversie al convertorului analogicnumeric), numită dispozitiv de reținere (memorare). De fapt, cele două dispozitive alcătuiesc un tot unitar, numit *dispozitiv de eșantionare și reținere*.

Conform celor precizate anterior, rolul dispozitivelor de eșantionare și reținere (DER), în cadrul SAD, este acela de a eșantiona semnalele corespunzătoare canalelor de intrare, provenind de la senzori și de a menține aceste valori constante pe toată durata conversiei analog-numeric. DER este un element indispensabil în SAD, apărând în toate variantele analizate. Frecvența de eșantionare maximă permisă a unui SAD depinde, în principal, de viteza de conversie a CAN din structura lui și de numărul de canale de intrare. În general, cerințele simultane de: rezoluție mare, număr de canale de intrare mare și utilizarea unui singur CAN, nu pot fi îndeplinite pentru mărimi rapid variabile în timp.

Schema de principiu a unui DER conține un comutator K și un condensator C (fig.16.10).

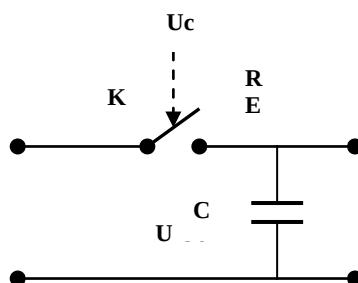


Fig. 16.10. Schema de principiu a unui DER.

Comutatorul este comandat de semnalul binar U_c , cu nivelul E (jos) pentru *închis* și nivelul R (sus) pentru *deschis* (fig.16.11). Condensatorul C , presupus ideal, se încarcă, pe durata corespunzătoare nivelului E la valoarea instantanee pe care tensiunea U , de la intrare, o ia la sfârșitul acestei durate. Valoarea respectivă este memorată pe intervalul de timp corespunzător nivelului R. Cele trei semnale: de intrare (U), de comandă (U_c) și de ieșire (U_e) sunt prezentate în figura 16.11. Durata nivelului R este mult mai mare decât durata nivelului E. Semnalul de comandă este periodic, de perioadă T_e , numită perioadă de eșantionare.

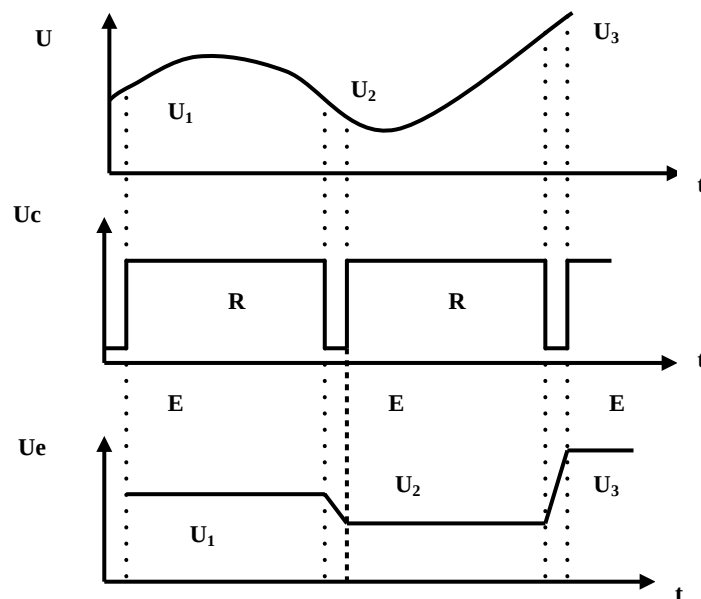


Fig. 16.11. Diagramele de semnale de intrare (U), comandă (U_c) și ieșire (U_e) pentru un DER.

Cuantizarea. A doua funcție îndeplinită de SAD o reprezintă conversia analognumerică a semnalului eșantionat. Aceasta este funcția principală a convertoarelor analognumerice propriu-zise. Caracteristica statică a unui CAN (partea de cuantizare) este prezentată în figura 16.12, prin dependența dintre valoarea cuantizată și valoarea eșantionată ale semnalului.

Valorile de ieșire sunt constrânse pe anumite niveluri, numite *niveluri de cuantizare*, iar trecerea de pe un nivel de cuantizare pe altul se face la depășirea de către semnalul de intrare (semnalul eșantionat) a unor niveluri, numite *niveluri de decizie*. Distanța dintre două niveluri succesive de cuantizare se numește *pas de cuantizare*. Dacă pasul de cuantizare este constant, atunci cuantizarea se numește *uniformă*, fiind cazul cel mai des întâlnit. De regulă, nivelurile de decizie se iau la mijlocul intervalelor de cuantizare, deoarece, în acest caz, eroarea de cuantizare este minimă. Pentru a demonstra acest lucru, fie $x_e(i)$, $i = 0, 1, 2, \dots$, valorile eșantionate ale semnalului x . Notăm $x_e = x_e(i)$ și cu $x_c = x_c(i)$ valoarea cuantizată a lui $x_e(i)$. Considerând $L_m \leq x_c \leq L_M$, unde L_m și L_M sunt limita inferioară, respectiv, limita superioară, ale domeniului de variație al valorilor lui x , problema cuantizării are drept scop determinarea unor niveluri de decizie d_j și a unor niveluri de cuantizare r_j astfel încât, dacă $d_j \leq x_e \leq d_{j+1}$, atunci eșantionul cuantizat să ia valoarea r_j (fig. 16.12).

Nivelurile de decizie și de reconstrucție sunt alese astfel încât să se minimizeze eroarea medie pătratică de cuantizare (între x_e și x_c):

$$\varepsilon_C^2 = M \int_{L_m}^{L_M} (x_e - x_c)^2 p(x_e) dx_e$$

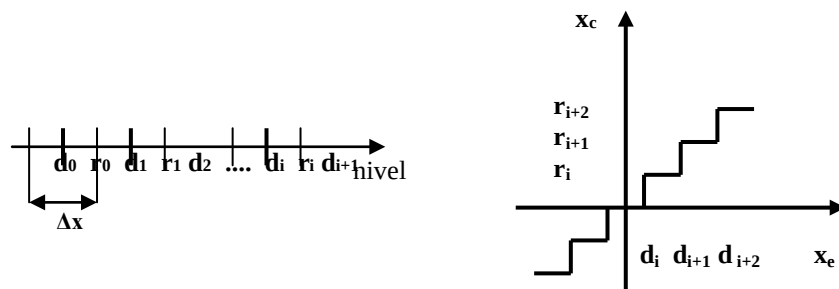


Fig. 16.12. Niveluri de decizie și de reconstrucție.

Rezultă:

$$r_j \approx \frac{1}{2} d^{j \oplus 1} \oplus d_j$$

După cuantizare urmează codificarea valorii cuantizate, de regulă în cod binar natural, complementar față de 2 sau BCD.

Convertoare analog-numerice. CAN reprezintă un element important al oricărui SAD, atât prin faptul că realizează conversia semnalelor analogice în semnale numerice, fiind nelipsit din orice structură de SAD, cât și prin faptul că timpul necesar conversiei A-N reprezintă componenta cea mai semnificativă a duratei de achiziție. Datorită cerințelor speciale care pot apărea în cadrul unor aplicații, există o mare varietate de metode și tipuri de scheme. Criteriile care conduc la alegerea unui CAN sunt: rezoluția, timpul de conversie, precizia, costul, codificarea ieșirii etc. De regulă, CAN realizează o comparație a tensiunii analogice de intrare U_i cu o tensiune de referință U_r .

Există mai multe criterii de clasificare a CAN. Astfel, după modul în care se face comparația dintre U_i și U_r , deosebim: CAN cu comparație directă (CAN paralel, CAN cu aproximații succesive etc.) și CAN cu comparație indirectă (cu conversie tensiune-frecvență, cu conversie tensiune-timp). După proveniența tensiunii U_r , deosebim CAN în circuit deschis (fără reacție), la care U_r este o tensiune de referință fixă, și CAN în circuit închis (cu reacție), la care U_r este tensiunea de reacție. Din prima categorie fac parte CAN paralel, CAN cu integrare etc., iar din a doua categorie fac parte CAN cu aproximații succesive, CAN cu compensare în trepte egale etc.

CAN cu comparație de tip paralel. CAN de tip paralel efectuează conversia foarte rapid și, din această cauză, se mai numesc convertoare flash. Un CAN paralel este alcătuit din trei niveluri, fără reacție: nivelul de divizare a tensiunii, nivelul de comparație și nivelul decodificator (fig. 6.13). Timpul de conversie este dictat de timpul de propagare prin comparatoare și prin porțile logice ale circuitului logic combinațional, fiind foarte scurt. În schimb, numărul de comparatoare folosite crește exponențial cu numărul de biți cerut de rezoluție. Din această cauză, CAN de tip paralel nu se pot utiliza în aplicații care cer rezoluții sporite. În schimb, ele sunt larg folosite în aplicații care cer viteze de achiziție foarte mari, ca de exemplu: prelucrarea numerică a semnalului video, prelucrarea semnalului radar. Timpul de conversie poate ajunge până la zeci de nanosecunde.

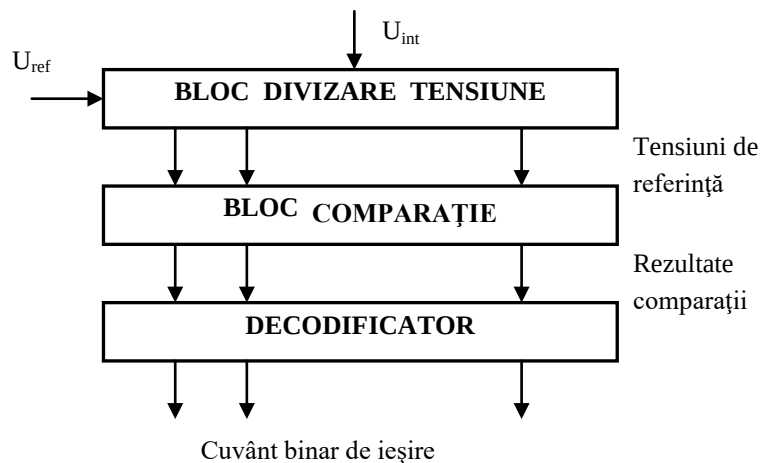


Fig. 16.13. Schema de principiu a unui CAN paralel.

CAN cu comparație de tip serie-paralel. CAN de tip serie – paralel păstrează calitățile CAN paralel (viteză de conversie mare), dublând practic rezoluția. El este alcătuit, în principiu, din două CAN de tip paralel conectate în serie. Evident, timpul de conversie este de câteva ori mai mare decât la CAN de tip paralel. Considerând numărul de biți n , par, principiul de funcționare al CAN de tip serie constă în următoarele etape:

- tensiunea de intrare U_{int} se convertește analog-numeric pe $n/2$ biți, prin metoda paralelă, obținându-se biții cei mai semnificativi ai conversiei globale ($b_0, b_1, \dots, b_{n/2-1}$);
- se face conversia numeric – analogică a secvenței obținute ($b_0, b_1, \dots, b_{n/2-1}$), rezultând o tensiune U_{IC} ;
- U_{IC} se scade din U_{int} , rezultând eroarea de cuantizare ΔU_C ;
- ΔU_C se amplifică cu $2^{n/2}$, iar rezultatul este convertit analog-numeric pe $n/2$ biți, tot prin metoda paralelă, obținându-se cuvântul binar ($b_{n/2}, b_{n/2+1}, \dots, b_{n-1}$), ceea ce reprezintă biții cel mai puțin semnificativi ai conversiei globale.

CAN cu aproximații succesive. Datorită faptului că acest tip de CAN realizează un bun compromis între viteza de conversie și rezoluție, el este unul dintre cele mai folosite convertoare în structurile SAD.

Principiul de funcționare al CAN cu aproximații succesive constă în compararea succesivă, bit cu bit, începând cu bitul cel mai semnificativ, a rezultatelor conversiilor parțiale (tensiunea de reacție U_r) cu tensiunea de intrare, U_i (fig. 16.14). Semnificația notațiilor din figură este următoarea: C-comparator, GT-generator de tact, RAS-registru de aproximații succesive, CNA-converter numeric-analogic, EN-ieșire numerică. Tensiunea de comparare U_r se formează pe baza biților validați în etapele anterioare. Pentru un convertor pe n biți vor fi n comparații ale celor două tensiuni.

Rezultatele parțiale se stabilesc în felul următor:

- inițial, conținutul RAS este zero;

- la începutul fiecărui pas, se pune 1 logic în bitul corespunzător de test al registrului de aproximații succesive, restul biților rămânând neschimbați față de pasul anterior. Primul pas corespunde bitului celui mai semnificativ, iar ultimul pas corespunde bitului celui mai puțin semnificativ;
- se compară tensiunea de reacție corespunzătoare pasului curent cu tensiunea de intrare. Dacă este mai mare decât tensiunea de intrare, în bitul de test se pune 0, iar dacă este mai mică, bitul de test rămâne 1;
- procedeul continuă până la testarea corespunzătoare bitului celui mai puțin semnificativ. În acest moment se poate considera că, la ieșirea CAN se află rezultatul conversiei.

Timpul de conversie al unui CAN cu aproximații succesive de n biți este nT , unde T reprezintă perioada tactului de lucru și poate ajunge până la câteva microsecunde.

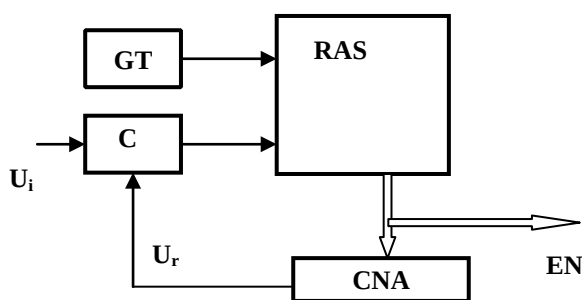


Fig. 16.14. Schema funcțională a CAN cu aproximații succesive.

CAN cu integrare. CAN cu integrare sunt mai lente, dar au rezoluția cea mai bună. Ele realizează conversia indirectă a tensiunii de intrare într-un număr, prin intermediul unui interval de timp. CAN de acest tip se construiesc în variante cu două, trei sau patru pante. În fig.16.15 este prezentată schema funcțională a unui CAN cu integrare cu două pante. Un astfel de convertor este fără reacție și funcționează în două faze. În prima fază, tensiunea de intrare U_i este integrată cu o pantă proporțională cu ea, pe un interval de timp fixat t_1 , de către blocul de integrare INT (fig.16.16). Valoarea la care ajunge ieșirea integratorului U_{int} este, de asemenea, proporțională cu U_i . În faza a doua, blocul INT integrează, cu pantă constantă, o tensiune de referință fixată U_r , de semn contrar cu U_i . Timpul de integrare pentru această fază t_x va fi proporțional cu U_i . Numărătorul reversibil NR are rolul de a stabili, pe de o parte, timpul de integrare t_1 , iar pe de altă parte, un număr EN, proporțional cu U_i , ca rezultat final. Blocul de comandă BC, prin secțiunile BC1 și BC2, are rolul de a asigura desfășurarea celor două faze ale conversiei analog-numerice bazată pe dubla integrare.

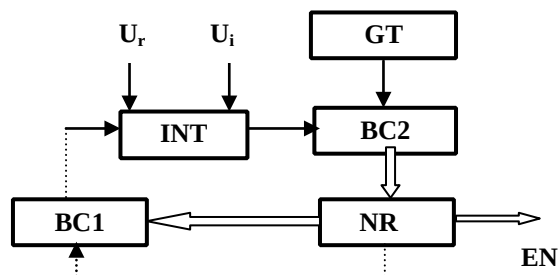


Fig.16.15. Schema funcțională a unui CAN cu integrare cu dublă pantă.

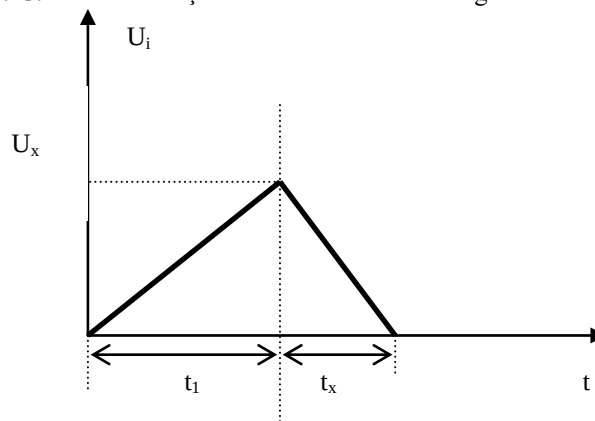


Fig. 16.16. Principiul de conversie analog-numerică prin integrare cu dublă pantă.

2. Transmiterea la distanță a datelor

2.1. Organizarea transmisiei de date în sisteme de comunicație punct la punct

În figura 16.18 se prezintă schema bloc funcțională a unui sistem de comunicație, având ca obiectiv transmiterea informației în timp și spațiu de la un punct, numit sursă (de informație) sau expeditor, la un alt punct, denumit utilizator sau destinatar. În mod particular, pentru un proces industrial sursa de informații poate fi un traductor, iar destinatarul un calculator de proces.

Schema din figura 16.18 nu este generală, în sensul că s-a considerat ca primă restricție natura electrică a semnalelor de intrare și ieșire, ceea ce implică necesitatea ca o sursă neelectrică de informație să posede un mecanism de conversie a informației într-un semnal electric variabil în timp, ce va fi denumit *semnal mesaj*. La rândul său, canalul de comunicație trebuie să permită transmiterea semnalului electric, dar natura sa poate fi diversă: pereche de fire metalice, fibră optică, canal radio ș.a. În plus, s-a considerat că mesajele sursă și utilizator sunt secvențe de simboluri binare (cazul cel mai întâlnit în situația în care realizarea echipamentelor se face prin apelare la tehnica de calcul).

La transmiterea prin canalul de comunicație poate apărea o degradare a semnalului din cauza perturbațiilor sau distorsiunilor provocate de tehnica de transmisie. Principalele cerințe pentru un sistem de comunicație sunt: 1) evitarea distorsiunilor și minimizarea efectelor perturbațiilor (cerința de precizie) și 2) o viteză de transmisie cât mai mare, pentru a minimiza costurile de ocupare a canalului (cerința de viteză).

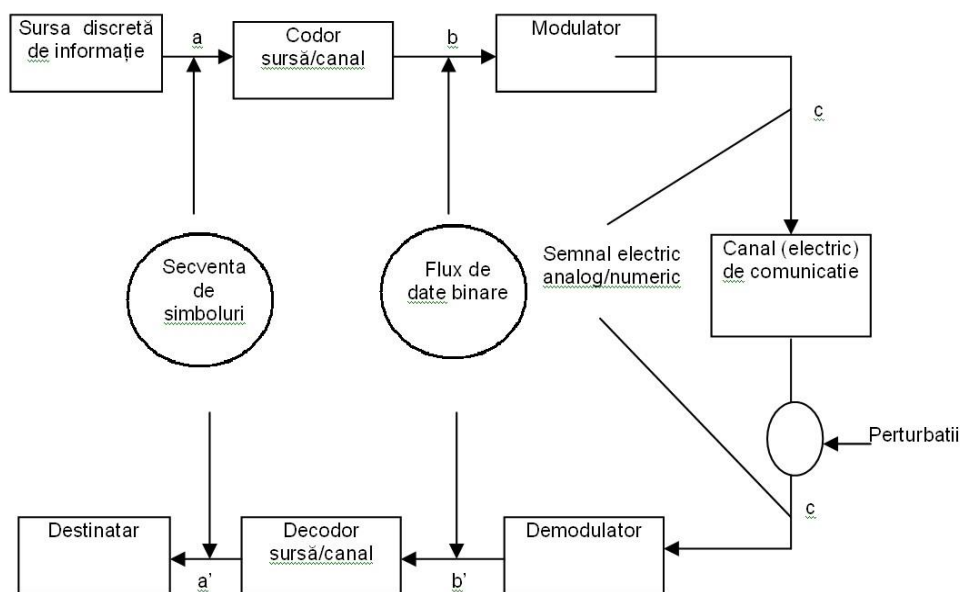


Fig.16.18. Schema de principiu a unui sistem de comunicație punct la punct.

Pentru a atinge prima cerință, emițătorul va prelucra mesajul inițial, pentru a avea o transmisie eficientă. Principalele operații efectuate sunt: amplificare, filtrare, modulare – ultima operație fiind esențială în adaptarea semnalului mesaj la caracteristicile canalului. Ea oferă totodată posibilități de reducere a efectelor perturbațiilor și de transmitere simultană a mai multor mesaje. Există două tipuri fundamentale de modulație: cu purtătoare continuă (de regulă sinusoidală) și cu purtătoare tren de impulsuri. În ambele cazuri, modificarea purtătoarei de către semnalul mesaj se poate face continuu sau discret, ultima procedură fiind preferată, în sensul că permite modularea directă a semnalelor discrete ale sursei. Totuși, folosind tehnici de discretizare în timp (eșantionare) și nivel (cuantizare), se pot transmite și semnale analogice prin tehnici discrete de modulare.

O altă procedură esențială în transmiterea de date o prezintă operațiile duale de codare (la sursă) și respectiv decodare (la destinatar). Blocul de codare are în componență două subansambluri: blocul de codare sursă (care transpune mesajul în alfabetul sursei) și blocul de codare canal (care transpune mesajul în alfabetul canalului). Prin tehnicile de codare, o secvență de simboluri capătă o anumită semnificație, anumite reguli semantice permițând depistarea la decodare a eventualelor erori apărute în timpul transmisiei și, în unele cazuri, corectarea acestora. Din păcate, tehnicile de codare/decodare necesită introducerea de simboluri suplimentare de control (redundanță), ceea ce duce la scăderea vitezei de transmisie în canal. Prin urmare, atingerea simultană a celor două obiective – eroare de transmisie cât mai mică și viteză de transmisie cât mai mare, – nu este posibilă. De aceea e necesară o soluție de compromis: se impune o anumită limită a erorii admisibile, după care se crește viteza de transmisie până la valoarea care nu duce la ieșirea din marja de eroare impusă. Trebuie menționat faptul că viteza de transmisie (mai exact debitul informațional, adică numărul de simboluri binare transmise într-o secundă) poate fi crescută pe de o parte prin utilizarea unui canal de bandă cât mai mare (fibra optică este canalul cu banda cea mai largă), iar pe de altă parte prin creșterea raportului semnal/zgomot (ceea ce implică un cost mai mare). Evident, pe un canal neperturbat (ideal) se poate transmite cu o viteză oricât de mare.

Totodată trebuie precizat faptul că schema din figura 16.18 este pur teoretică, fără aplicabilitate industrială, pentru că privește unilateral transmisia de date. În practică se impune

o soluție prin care datele circulă în două sensuri, între două echipamente terminale de transmisie de date (ETTD). Figura 16.19 prezintă schematic un astfel de sistem.

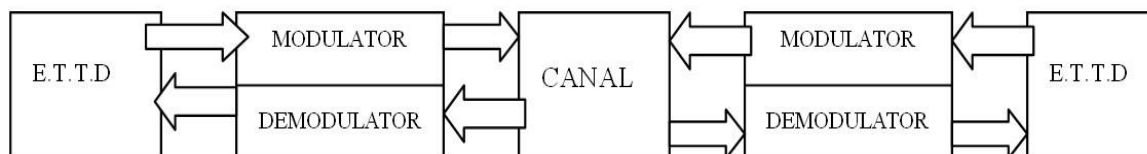


Fig.16.19. Transmisie punct la punct bidirecțională – schemă bloc.

În schema din figura 16.19 am presupus că blocurile modulator și demodulator au fost asociate într-un bloc funcțional denumit modem, iar ETTD înglobează și blocurile de codare/decodare. Un astfel de sistem de comunicație poartă denumirea de sistem de comunicație punct la punct sau post la post, aspectul de distanță putând fi marcat prin noțiunile de post local și post distant. Sistemele de comunicație punct la punct constituie doar o etapă în evoluția sistemelor de comunicație, care s-au dezvoltat sub formă de sisteme de comunicație multipunct și de rețele de transmisie de date, în care numeroase terminale pot efectua schimburi complexe de informații prin sisteme standardizate de interfață fizică și logică. Pentru a permite o structurare modulară, flexibilă și adaptabilă în permanență la progresele tehnologice, dar și pentru a asigura interoperabilitatea între diferitele produse existente pe piață, s-au făcut eforturi de standardizare care s-au concretizat spre sfârșitul deceniului nouă din secolul trecut prin lansarea de către Organizația Internațională de Standardizare (ISO) a unui model de referință cu 7 niveluri pentru stabilirea cerințelor de asigurare a comunicației între două echipamente terminale de transmisie de date cunoscut sub numele de “Modelul de referință de bază pentru interconectarea sistemelor deschise”[6], a cărui schemă este prezentată în figura 16.20.

7	APLICAȚIE
6	PREZENTARE
5	SESIUNE
4	TRANSPORT
3	REȚEA
2	LEGATURĂ DE DATE
1	FIZIC

Fig.16.20. Schema de principiu a modelului de referință ISO-OSI.

În filosofia ISO – OSI, noțiunea de sistem “deschis” se referă la un ansamblu de mai multe unități de calcul, cu software-ul asociat, cuprinzând terminale periferice, procesele fizice și mijloacele de transport al informației care se constituie într-un tot autonom capabil să facă și/sau transferuri de comunicație. Diferite sisteme “deschise” unul altuia în privința transferului de informații sunt conectate prin suporturi fizice (nivelul 1 - fizic). Indiferent însă de complexitatea structurii de comunicație, esențială rămâne asigurarea legăturii de date dintre două stații, realizată la cel de al doilea nivel, denumit *legătură de date*. Nivelul *legătură de date* furnizează mijloacele funcționale și procedurale necesare stabilirii, menținerii și eliberării liniei de transmitere a datelor, precum și transferul unităților de date de serviciu-legătură. O

conexiune de legătură de date se realizează cu ajutorul unei conexiuni fizice între două posturi (noduri) fără noduri intermediare. Modul în care operează cele două noduri este guvernat de o procedură de comandă a legăturii, numită și protocol de comunicație pe legătură (PCL).

2.2. Protocoale de comunicație pe legătura de date

Clasificarea protocoalelor de comunicație pe legătura de date. Există două categorii de PCL: protocoale orientate pe caracter (BCP - Byte Control Protocols) și protocoale orientate pe bit (BOP - Bit Oriented Protocols). BCP, apărute mult înainte de lansarea modelului de referință, se bazează pe o transmisie cu sincronizare de cuvânt (de regulă un octet). Mesajul constă dintr-un șir de octeți, fiind delimitat de o secvență de sincronizare care determină poziția primului octet în mesaj - antetul. Antetul conține informații auxiliare care identifică adresa mesajului (sursă sau destinație), tipul acțiunii de control, o confirmare (pozitivă sau negativă) asupra recepției fără eroare a mesajului. Acțiunile de control permit inițializarea sau dezactivarea unei stații secundare, acceptarea sau refuzul unui bloc de mesaje anterior, sau secvențe de transfer. Urmează câmpul de text, care conține toate datele ce trebuie transmise. Textul poate fi compus din caractere (cuvinte de cod) sau poate fi transparent la acest cod (date binare, date zecimale, date în virgulă mobilă) - situație care se impune în funcție de asigurarea transparenței de cod. În figura 16.21 este prezentată structura celui mai cunoscut și utilizat PCL-BCP, și anume BISYNC, sau BSC (Binary Synchronous Communication).

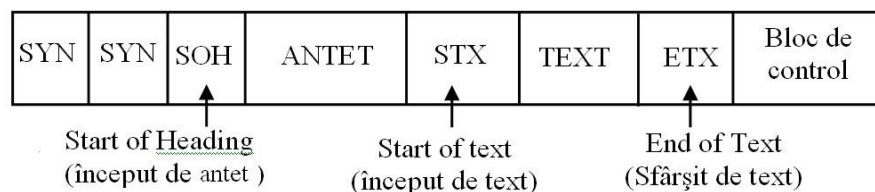


Fig.16.21. Formatul unui mesaj BSC.

Protocoalele orientate pe bit, apărute după 1980, sunt mai puternice și au grad pronunțat de universalitate. Mesajele BOP se transmit în formate fixe, fiind tratate ca șiruri de biți. Caracteristicile comune pentru BOP sunt: independența codurilor, configurarea liniei, folosirea semnificației poziției caracterelor de control, posibilitatea de operare duplex sau semi-duplex, realizarea transparenței prin metoda umplerii cu zero, controlul erorilor pentru întregul format. BOP utilizează câteva caractere specifice de control pentru delimitatori de început (FLAG) și de sfârșit (FLAG, ABORT) al mesajului. În figura 16.22 se prezintă formatul pentru cel mai frecvent utilizat protocol BOP și anume HDLC (High Level Data Link Control) [7].

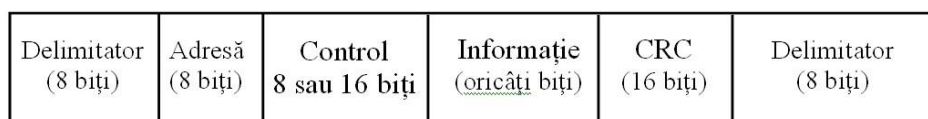


Fig.16.22. Formatul unui mesaj HDLC.

Funcțiile protocoalelor de comunicație. În legătura punct la punct, cele două stații pot fi în situația: una sursă, una receptor. Pe de altă parte, pentru a gestiona legătura, una dintre stații trebuie să asigure funcțiile primare, și este stația principală (master), iar cealaltă este stația

secundară (slave). Într-o legătură multipunct, există mai multe stații secundare și una sau mai multe stații principale.

Asociația între statutul de primar/secundar și cel de sursă/receptor se face în mai multe feluri:

- dacă stația primară este sursă, iar cea secundară este receptor, se spune că modul de transfer al informației este de tip invitație la recepție (în l. engleza *selecting*);
- dacă stația primară este receptor, iar cea secundară este sursă, modul de transfer al informației este de tip invitație la emisie (în l. engleza *polling*).

Termenul de polling s-a extins pentru transferul de informație în configurațiile de rețea cu o singură stație principală și mai multe stații secundare. Această situație se întâlnește în cele mai moderne arhitecturi de sisteme de achiziție de date de tip magistrală de câmp, la care stațiile secundare sunt fie traductoare inteligente, fie dispozitive compacte ce gestionează mai multe intrări și ieșiri din proces, având implementată și funcția de comunicare

În figura 16.23 se prezintă trei configurații posibile ale legăturii punct la punct:
a) simetrică; b) asimetrică; c) echilibrată.

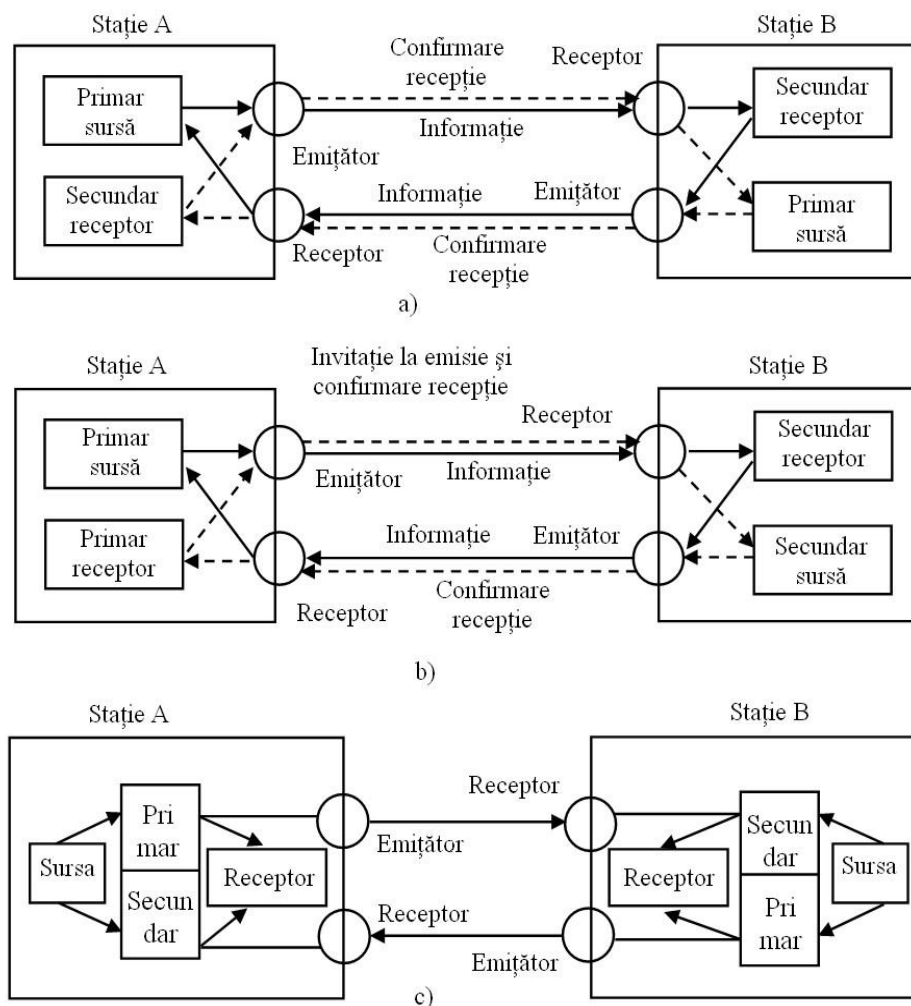


Fig.16.23. Trei configurații de comunicație punct la punct.

Exemplul din figura 16.23.a reprezintă o combinație de două legături simetrice de tip invitație la recepție, în care sursa e întotdeauna asociată cu o stație primară și receptorul întotdeauna cu o stație secundară. Această configurație, deși renunță la invitații la emisie, nu poate fi utilizată în legături multipunct, care sunt prin definiție asimetrice la un moment dat: o singură stație e primară, restul secundare. În figura 16.23.b se prezintă o configurație asimetrică, produsă prin asocierea unei legături cu invitație la emisie (linie plină) cu o legătură cu invitație la recepție (linie punctată). În acest fel, funcțiile primare sunt întotdeauna la o stație, cele secundare la celelalte. În cazul legăturii echilibrate (fig 16.23.c) informația poate fi transmisă oricum, fie ca o comandă, fie ca un răspuns. Avantajul imediat este acela că se pot asocia în același bloc mesaj (cadru) atât informația curentă, cât și confirmarea de recepție a unui cadru anterior [8].

Gestionarea legăturii cu protocolul HDLC. HDLC a fost ales de ISO ca BOP, având posibilitatea de a asigura atât transmisia asincronă cât și sincronă, dar evident e preferat în transmisii sincrone sau autosincrone. Transmisia în HDLC se poate face în trei moduri diferite:

1. *Modul normal de răspuns (NRM - Normal Response Mode)* se poate utiliza în configurații punct la punct sau multipunct, în ambele cazuri cu o singură stație primară. Stațiile secundare sunt invitate la emisie de stația primară. După începutul transmisiei, linia se menține în stare activă până când stația secundară transmite în ultimul cadru indicația că va opri emisia (bitul 5 din cuvântul de control F=1);
2. *Modul de răspuns asincron (ARM - Asynchronous Response Mode)* se mai numește și mod de funcționare autonomă; folosit de asemenea în transmisii punct la punct sau multipunct cu o singură stație principală. În modul autonom, are loc o retransmisie a stației secundare dacă primarul nu a transmis confirmarea pozitivă într-un interval de timp dat;
3. *Modul asincron echilibrat (Asynchronous Balanced Mode)* se utilizează numai în configurații punct la punct, comunicația având loc între două stații combinate.

2.3. Soluții standard pentru echipamente de comunicație industrială

Deoarece la nivelul *fizic* se asigură circuitele de legătură între două posturi ce comunică și procedurile prin care mesajul emis de sursă ajunge la receptor, la acest nivel vom regăsi toate mijloacele și tehnicile referitoare la alegerea canalului, la prelucrarea semnalelor, la prelucrarea mesajelor și respectiv la organizarea transmisiei. Întrucât operațiile de codare, decodare și organizare a transmisiei sunt intim legate de tipul de echipament terminal de prelucrare a datelor (ETPD) și de natura operației, standardizarea la nivelul fizic vizează doar echipamentul ce face interfața între echipamentul terminal de prelucrare a datelor și canalul de comunicație. Acest echipament, numit de obicei echipament de terminare a circuitului de date, este în fond cel care asigură funcțiile de prelucrare de semnal prin modulare/demodulare, deci un *modem*. CCITT (acum ITU) a întreprins standardizarea unei game largi de modemi astfel încât să faciliteze legăturile de date. Dar lista modemurilor e departe de a fi închisă, pentru că orice progres în tehnologie se reflectă într-o nouă realizare. Totuși, există doi parametri care fixează oarecum tipul de modem: viteza de transmitere și suportul fizic de transmisie. Pentru sistemele de achiziție de date din mediul industrial se folosesc în special linii bifilare torsadate (în special pentru rețele locale), deoarece asigură cel mai bun raport performanță/preț. Pot fi luate în considerație (pentru transmisii pe distanțe de peste 1200 m) și alte modalități de transmisie de tip linie telefonică: curenți purtători (canalul “grup primar”), linii specializate și chiar rețeaua telefonică comutată. Pentru transmisiile asincrone, vitezele uzuale pe aceste suporturi de

comunicație sunt: 300, 600, 1200 bit/s, iar pentru transmisiile sincrone: 600, 1200, 2400, 4800, 9600, 19 200, 48 000, 56 000, 64 000, 72 000, 128000 și 144000 bit/s.

Nu numai modemurile trebuie să corespundă unor standarde, ci și circuitele ce asigură schimburile de semnale între terminal (ETPD) și modem (ETCD). O interfață de acest tip implică standardizarea în patru categorii de caracteristici: mecanice, electrice, funcționale, procedurale, necesare realizării următoarelor faze ale transmiterii de date punct la punct:

- stabilirea circuitului, necesară atunci când linia nu e alocată în permanență legăturii de date;
- inițializarea procesului de transmitere, constând în adaptarea modemului de linie, și care include: emisia în linie a purtătoarei, detecția la cealaltă extremitate a liniei, sincronizarea ceasurilor de emisie și recepție, declanșarea eventualelor blocuri de egalizare sau anulare de ecou;
- menținerea legăturii pe toată durata de transmisie, până când una din extremități ia decizia de deconectare;
- eliberarea circuitului de date.

Caracteristicile mecanice se referă la tipul de conectare utilizat: numărul de pini (15, 25, 34, 37), atribuirea de pini a circuitului de interschimb, arhitectura conectorului, modul de montare a conectorului în rac. Dintre interfețele mecanice recomandate de ISO, menționăm ISO 4963, ISO 2110, conector cu 25 pini, ISO 2593, conector cu 34 pini.

Caracteristicile electrice fac obiectul mai multor standarde și reprezintă și singura structură de transmisie directă a semnalelor binare în banda de bază (fără modulație), sub denumirea generică de interfață electrică.

Caracteristicile funcționale se referă la tipurile de semnale care asigură schimbul de informații, grupate în patru categorii: de masă, de date, de sincronizare, de comandă. Principalul standard care normalizează aceste circuite este avizul CCITT V24.

Caracteristicile procedurale se referă la setul de proceduri necesare folosirii circuitelor de interschimb. Exemple:

Avizul V.28/RS-232C. Informația binară este vehiculată sub forma asimetrică (neechilibrată), pe distanțe de până la 15 m și viteze până la 20 kbiți/s. Fiecărui circuit de interfață îi corespunde un fir al cărui potențial se măsoară în raport cu o masă comună (masa de semnalizare) după schema din figura 16.24.

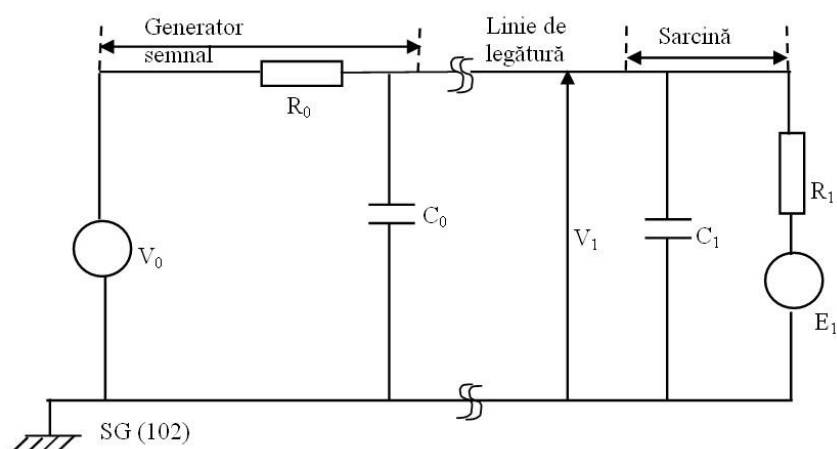


Fig.16.24. Schema unui circuit de interfață cu terminație unică.

Este deci un circuit cu terminație unică, bipolar, la care "0" logic este reprezentat de un semnal: $5 \square V_0 \square 25V$, iar "1" logic de un semnal: $\square 25 \square V_0 \square 5V$. Impedanța de sarcină are o rezistență de $3000 \square R_s \square 7000$ și o capacitate $C_s \square 2500pF$. La recepție nivelurile semnificative de funcționare sunt: $V_i \square 3V$ pentru starea binară "1", și respectiv $V_i \square 3V$ pentru "0". Fronturile trebuie să aibă o durată de cel mult 1 ms, (sau 3% din durata semnalului de ceas), dar cu o viteză de creștere sub 30 μs .

EIA RS-485. Standardul RS-232C se referă doar la transmisii punct la punct. Standardele RS 423 A și RS 422A pot asigura și transmisii multipunct, dar în sensul că există întotdeauna un singur emițător și un număr mai mare de receptoare care eventual pot primi simultan același mesaj (maximum 10).

Standardul RS-485 (deocamdată fără echivalent CCITT) este cel mai performant standard de interfață electrică, asigurând în mod real posibilitatea unei conectări în rețea (multipunct). Standardul consideră că un emițător poate avea o sarcină totală de maximum 32 USS și o rezistență terminală nu mai mică de 60Ω . În cazul în care emițătorul e în stare activă, el apare ca o sursă de tensiune cu impedanță joasă care produce în cablu o tensiune diferențială, în gama 1,5 V...5 V, astfel încât pentru starea logică "1" V_a să fie negativ în raport cu V_b , iar pentru starea logică "0" pozitiv. În regim dinamic, fronturile (crescătoare și descrescătoare) nu trebuie să depășească 30% din durata totală a impulsului t_B , iar variația față de valoarea de vârf nu trebuie să depășească 10%. Potențialul terminalului A este negativ față de B pentru valoarea logică "1". Zona de nedeterminare în decizie nu depășește 0,4 V.

Interfața RS-485 asigură condiții excelente pentru transmiterea semnalului în banda de bază, în cadrul unor rețele locale. Frecvența cu care se emit/recepționează biții de informație poate ajunge la 10 Mbit/s, cu condiția ca tranzițiile liniei să fie de maxim 20 ns. Aceasta depinde de capacitatea impedanței terminale, care se încadrează în jurul 100 pF pentru o rezistență de sarcină de maxim 54 $\square$. Se poate astfel asigura amplasarea de până la 6 stații combinate pe distanța de 2m, fiecare având o capacitate de 15 pF (2 m corespund cu aproximație distanței parcurse de un front de undă cu timpul de creștere de 20 ns).

3. Utilizarea magistrelor de câmp pentru conducerea proceselor

3.1. Caracteristici ale magistrelor de câmp

Sistemele de conducere automată sau sistemele de achiziție și distribuție de date sunt alcătuite, în marea majoritate a cazurilor, din dispozitive primare de măsurare/acționare, amplasate în proces, și unul sau mai multe elemente secundare de procesare, cu rol de prelucrare și administrare a datelor primare. Modul în care sunt amplasate unitățile de procesare definește două clase de sisteme.

Prima clasă – sisteme centralizate de conducere - cuprinde sistemele ale căror unități de procesare sunt grupate funcțional și constructiv într-un „punct dispecer” sau într-o cameră de comandă.

A doua clasă – sisteme distribuite de conducere – cuprinde sistemele la care o parte dintre sarcinile de procesare ale nodului dispecer sunt transferate către dispozitivele de măsurare/execuție distribuite în proces.

Distribuirea resurselor de procesare hardware și software către elementele din sistem, distribuite în proces, a creat o nouă problemă și anume aceea a comunicației între componentele sistemului. Era necesară realizarea unei „rețele” de comunicații care să permită vehicularea datelor între dispozitivele din sistem. Caracteristicile funcționale și constructive ale unei astfel de rețele erau: fidelitatea datelor, imunitate la zgomotele și perturbațiile specifice mediului industrial, viteză de lucru cât mai mare, flexibilitate la extinderea sistemului, instalare și întreținere ușoare și, nu în ultimul rând, cost redus.

Rețeaua de comunicații care satisface aceste cerințe și care este utilizată din ce în ce mai mult în sisteme distribuite de conducere poartă numele de magistrală de câmp (fieldbus). Denumirea sintetizează bine funcționalitatea. Astfel, „magistrală” sugerează un tronson principal de comunicație la care să fie conectate toate componentele sistemului. Magistrala este „de câmp” pentru că se întinde pe întreaga arie geografică a procesului, uneori în spații închise, alteori în câmp deschis.

Schema bloc funcțională a unui sistem de conducere „alinat” cu magistrala de câmp este prezentată în fig. 16.25.

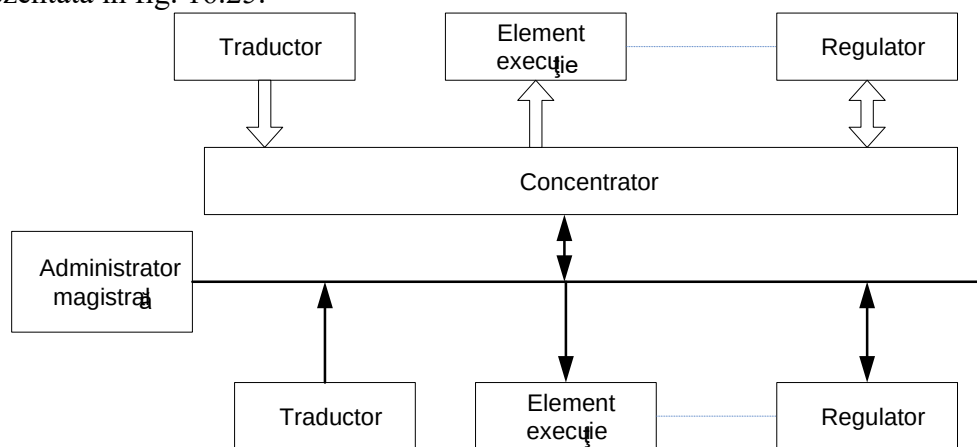


Fig.16.25. Sistem de conducere cu magistrală de câmp. Dezvoltarea și perfecționarea magistralelor de câmp au impus, la rândul lor, anumite condiționări ale sistemului de conducere, care a trebuit să devină compatibil cu un anumit tip de magistrală.

Împrăștierea geografică a componentelor sistemului determină dispozitivele din teren să lucreze independent. Integrarea lor în sistem se face prin transfer de informație efectuat prin intermediul magistralei de câmp. Dată fiind poziția lor într-o rețea de comunicație, elementele componente ale sistemului poartă numele generic de noduri.

Utilizarea magistralei de câmp determină o structură ierarhizată de tip Master-Slave a sistemului, cu două sau mai multe niveluri. Pe fiecare nivel ierarhic superior se află un singur nod Master activ, în timp ce pe nivelurile inferioare se află nodurile Slave. Pentru a ilustra alcătuirea unui sistem de conducere „alinat” la magistrala de câmp, în figurile 16.26 și 16.27 sunt prezentate schemele bloc ale unor sisteme cu două, respectiv mai multe niveluri ierarhice.

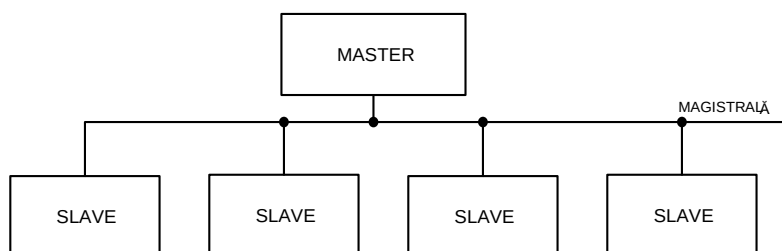


Fig. 16.26. Sistem ierarhizat de conducere cu două niveluri.

Nodurile aflate pe niveluri intermediare au funcționalitate dublă, Slave față de nivelul superior și Master față de nivelul inferior.

Pe nivelul superior sunt amplasate mai multe noduri Master interconectate într-o rețea separată. Aceasta înseamnă că sistemul poate fi alcătuit și într-o structură multi-Master, folosind o strategie independentă de conectare a nodurilor Master de pe același nivel, astfel încât în orice moment să fie activ doar un singur Master. Spre exemplu, nodurile Master pot fi conectate într-o structură de tip „token-bus” sau „token-ring”. Protocolul de funcționare a acestei structuri îl desemnează ca Master activ pe posesorul token-ului și definește modalitatea de transfer a token-ului. O astfel de soluție este utilă atunci când diverși algoritmi de conducere sunt implementați pe noduri independente sau atunci când există mai multe dispozitive Master care lucrează în regim de rezervare.

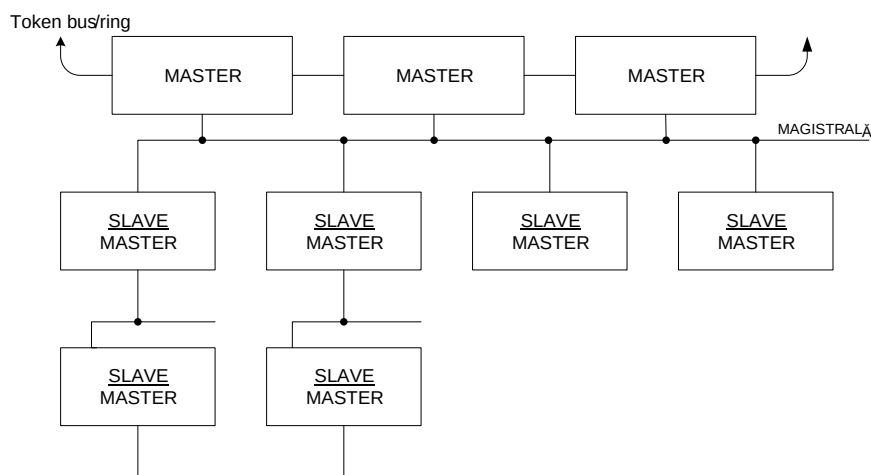


Fig. 16.27. Sistem ierarhizat de conducere cu mai multe niveluri.

În schemele prezentate sunt folosite două tipuri de noduri ce fac parte din structura funcțională a sistemului: noduri Master, cu rol de administrare/procesare, și noduri Slave, cu rol de preluare/distribuție de informație primară. Pentru buna funcționare a sistemului în condiții grele de mediu sau pe arii extinse a fost definit un al treilea tip de nod: nod repetor. Nodurile repetitoare sunt dispozitive proprii rețelei de comunicații, cu rol de amplificatoare de semnal, și nu sunt implicate în operațiile de conducere.

3.2. Comunicația pe o magistrală de câmp

Magistrala de câmp funcționează ca o rețea de comunicație pe baza modelului de referință ISO – OSI, care a fost prezentat mai sus (v. fig. 16.20). Magistralele de câmp folosesc doar trei dintre cele șapte niveluri și anume nivelul fizic (1), nivelul legăturii de date (2) și nivelul aplicație (7). Uneori peste nivelul 7 se adaugă un nivel utilizator.

Infrastructura magistralei de câmp (nivelul 1 al modelului ISO – OSI). Infrastructura rețelei cuprinde mediul de comunicație și dispozitivele de conectare la noduri și este construită pornind de la modul de codificare cu semnale a valorilor digitale de transmis.

În cele ce urmează va fi prezentat un exemplu de magistrală de câmp care vehiculează date pe cablu torsadat de cupru. Transmisia datelor se face în mod sincron, pilotată de un semnal de tact. Codificarea valorilor binare se face diferențial, adică polaritatea unuia dintre fire în raport cu polaritatea celuilalt dictează valoarea logică a bitului transmis.

Modul de codificare valoare binară/semnal electric și condițiile în care magistrala poate lucra ca o rețea multipunct sunt definite de standardul EIA RS 485. Structura constructivă a unui segment de magistrală realizat pe baza acestui standard este prezentată în figura 16.28. Pentru o funcționare corectă, standardul definește valori limită de curent (unități de sarcină) pe care fiecare nod le poate absorbi de pe magistrală.

În aceste condiții, magistrala poate fi construită pentru două regimuri de lucru:

□ sincron – cu semnalul pentru date și semnalul pentru tact vehiculate separat, pe perechi distincte de fire; □ autosincron – cu semnalele de date și de tact transmise pe aceeași pereche de fire. Existența celor două moduri de lucru permite un domeniu larg de opțiuni ale raportului viteză de transmisie/distanță într-o gamă variată de aplicații.

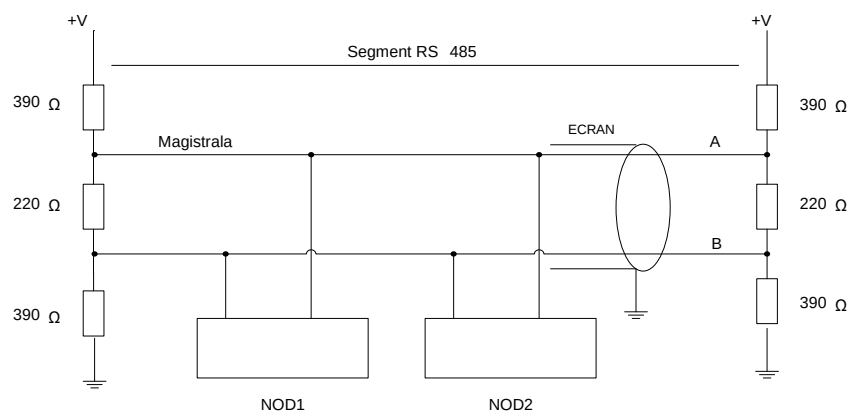


Fig. 16.28. Definirea unui segment de magistrală RS 485.

De exemplu, modul sincron de lucru este ilustrat în figura 16.29.

Semnalul util de pe cablul de date este codificat/decodificat folosind semnalul de sincronizare transmis pe cablul de tact. Astfel, semnalul de date se modifică pe frontul căzător al tactului și este eșantionat pentru preluare pe frontul crescător al tactului.

Modul sincron este dedicat aplicațiilor care necesită viteze mari de transmisie pe distanțe relativ scurte. Astfel, pot fi realizate viteze cuprinse între 500 kbit/s și 2,4 Mbit/s pentru distanțe de până la 30 m.

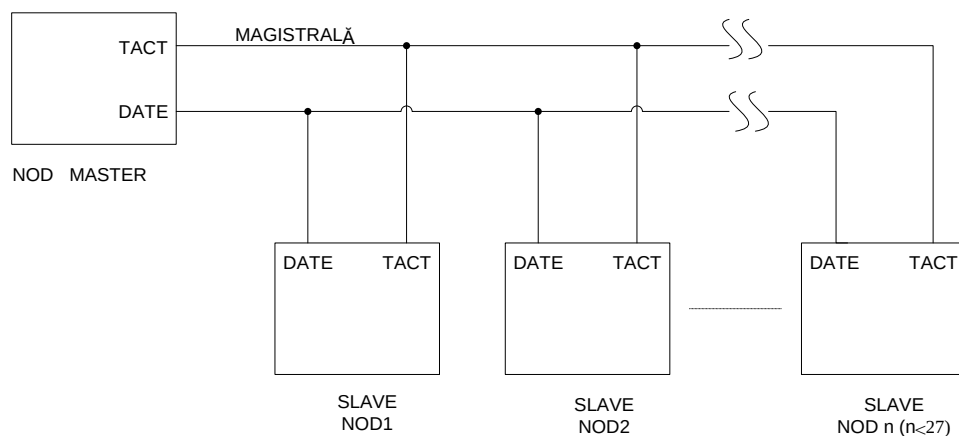


Fig. 16.29. Magistrală de câmp în mod sincron.

Protocolul legăturii de date pentru magistrala de câmp (nivelul 2 din modelul ISO-OSI). Operațiile specifice realizate pe acest nivel sunt: gestionarea comunicației multipunct între un nod Master și mai multe noduri Slave, împachetarea mesajelor preluate de la nivelul 7 în cadre ce pot fi transferate nivelului fizic, asigurarea securității datelor. De-a lungul timpului au fost definite protocoale care realizau operațiile caracteristice legăturii de date în diverse moduri.

Un protocol care realizează bine, eficient și „fiabil” legătura de date este SDLC (Synchronous Data Link Control) [9].

Protocolul de mesaj (nivelul 7 din modelul ISO-OSI). Protocolul de mesaj realizează interfața de comunicare task la task între un nod Master și mai multe noduri Slave, utilizând o structură comandă/răspuns (nodul Master emite comenzi, iar nodurile Slave transmit răspunsuri în consecință). Pe acest nivel, noțiunile comandă și răspuns au sensul unei conversații „funcționale” între dispozitivul central de administrare și subsistemele conduse. Interfața este construită pe nivelul 7 (aplicație) al modelului ISO-OSI, folosind cadre de informație pentru transferul de mesaje.

Protocolul de mesaj lucrează într-un mediu multitasking. Comanda este definită ca un mesaj transmis de un task de pe nodul Master către un task de pe un nod Slave, prin intermediul magistralei. Răspunsul este definit ca un mesaj transmis de un task de pe un nod Slave către un task de pe nodul Master, ca replică la o comandă recepționată.

În derularea protocolului de mesaj pot să apară o serie de erori: erori iremediabile de protocol sau pierderi de taskuri. În toate cazurile, protocolul are un mecanism de detectare și tratare a erorilor.

Formatul mesajului depinde de tipul de magistrală utilizat și de informația care este transferată. Există, totuși, anumite elemente constitutive comune majorității mesajelor, de care de altfel depinde calitatea conversației: adresa nodului Slave, adresa taskului de pe nodul Slave, adresa taskului de pe nodul Master, lungimea mesajului, tipul mesajului (comandă sau răspuns). Pentru a fi trimis în rețea, un astfel de mesaj este transferat protocolului legăturii de date și plasat în câmpul informație al unui cadru.

Configurarea aplicației (nivelul utilizator). Nivelul utilizator, pe care îl vom numi nivelul 7+ implementează strategiile de conducere pentru zona de proces în care activează

dispozitivul. Altfel spus, fiecare modul hardware care gestionează activități locale are o dublă funcționalitate: nod în rețeaua definită de magistrala de câmp și sistem local de conducere. Ca nod în rețea, dispozitivul transmite/recepționează mesaje asamblate în cadre de către protocolul legăturii de date. În acest fel, dispozitivul transmite informații sau execută comenzi în cadrul sistemului distribuit de conducere.

Tot pe nivelul utilizator este implementat serviciul de descriere dispozitiv (DD), care definește dispozitivul prin caracteristicile funcționale și constructive. DD permite sistemului de conducere să identifice dispozitivul din punct de vedere funcțional. DD conține toate variabilele de configurare și modurile de funcționare ale dispozitivului. Folosirea acestui tip de identificare funcțională asigură interoperabilitatea în sistem, prin aceea că pot fi folosite module de la diverși producători care au aceeași DD. Totodată, DD reprezintă un identificator pe baza căruia dispozitivul poate fi apelat de către Master.

3.3. Integrarea magistralei de câmp într-un sistem informatic

Sistemele de conducere gestionează procese industriale din diverse domenii de activitate (petrochimie, metalurgie, fabricarea hârtiei sau a medicamentelor etc.). În oricare dintre cazuri nivelul de competență al sistemelor de conducere este cel tehnic/tehnologic, adică derularea la parametri nominali a activității de producție. În același timp, la nivelul unității de producție există multe alte informații care contribuie la buna desfășurare a activității și care sunt gestionate de servicii specializate, asistate de rețele informatice proprii. Agregarea tuturor acestor resurse informatice generează ceea ce specialiștii numesc rețea de întreprindere (Enterprise Network).

În funcție de sectorul industrial de activitate, de mărimea companiei, de diversitatea operațiilor de efectuat, de nivelul tehnic al utilajelor existente, forma și complexitatea rețelei de întreprindere variază. Rețelele de întreprindere pot fi considerate colecții de LAN (Local Area Network), WAN (Wide Area Network), MAN (Metropolitan Area Network) sau combinații ale acestora, în care, pentru vehicularea informației, se folosește o gamă largă de protocoale de comunicație. Unele dintre aceste protocoale sunt folosite simultan pe aceeași platformă. Datele vehiculate sunt diverse și provin de la servicii dedicate: producție, întreținere, aprovizionare, desfacere, financiar, administrativ etc. Mai mult, odată cu extinderea utilizării internetului, numărul aplicațiilor ce pot lucra în rețea continuă să crească.

Structura unei rețele de întreprindere este prezentată în figura 16.34. Rețelele construite la nivelul de conducere a procesului pot fi de diverse tipuri. Fiecare zonă de proces va fi deservită de un sistem de conducere care să genereze performanțe și eficiență maxime. Aceasta înseamnă, însă, magistrale locale și protocoale diferite. Pentru interconectarea lor în rețeaua de întreprindere se folosesc dispozitive specializate de adaptare (gateway).

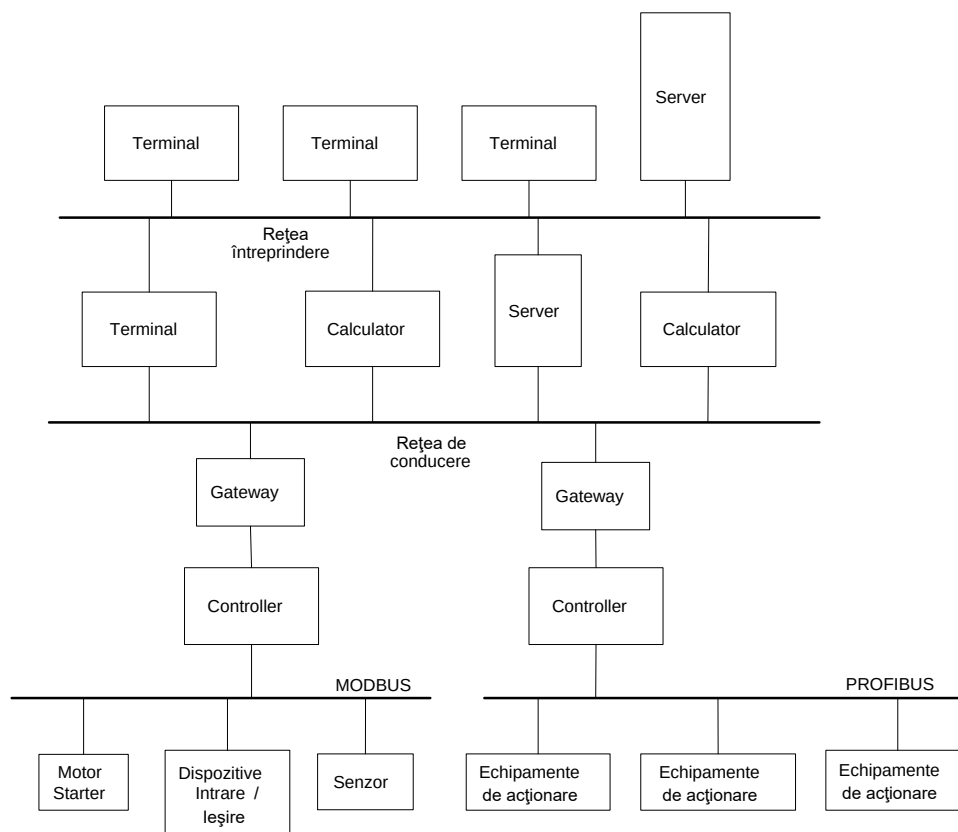


Fig. 16.30. Rețea de întreprindere cu magistrale de câmp la nivelul de conducere proces.

II. Automate programabile

1.1. Generalități ale automatelor programabile.

Automatul programabil este un dispozitiv bazat pe microprocesor care are drept scop principal monitorizarea constantă a stării sistemului și în funcție de aceasta și de programul introdus de utilizator să execute acțiunile necesare pentru a controla și menține performanțele sistemului ca în figura 1.1.

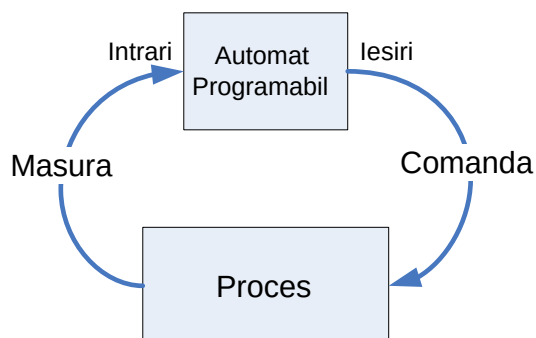


Fig. 1.1. – rolul automatului în automatizare

Automatul programabil poate fi folosit pentru a realiza simple acțiuni repetitive sau poate fi integrat împreună cu alte automate și computere într-o rețea pentru a putea comanda sisteme complexe.

1.1.2. Structura unui automat programabil.

Automatele programabile au în general o structură modulară pentru a se putea mai ușor configura cerințelor variate ale sistemelor ce urmează a fi automatizate. Modelul simplificat al unui automat programabil este cel din figura 1.2.

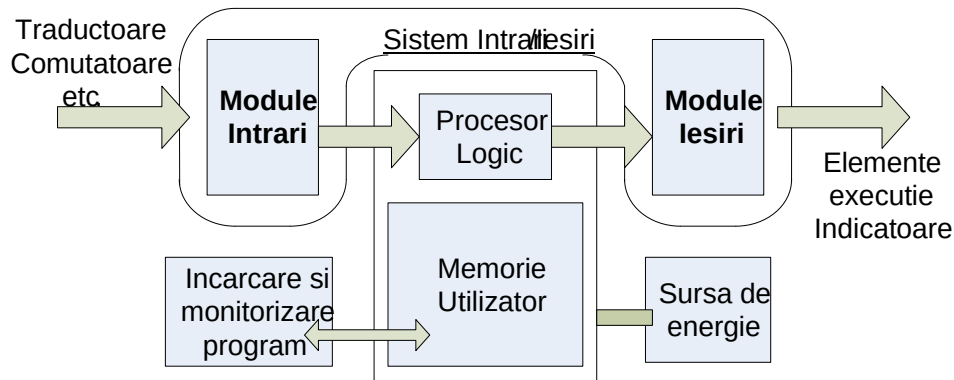


Fig. 1.2. – Modelul simplificat al unui automat programabil.

Modulele de intrare convertesc semnalele de nivel înalt care vin din dispozitivele aflate în câmp la semnale de nivel logic pe care procesorul le poate interpreta în mod direct. Procesorul Logic citește aceste semnale și decide ce stare trebuie să aibă ieșirile, bazându-se pe logica programului utilizator. Modulele de ieșire convertesc semnalele de nivel logic de la procesor în semnalele de nivel înalt necesare pentru comanda dispozitivelor din câmp.

Încărcarea programului și monitorizarea lui se face prin modulul specializat, iar energia electrică necesară calculelor este preluată din sursa de energie.

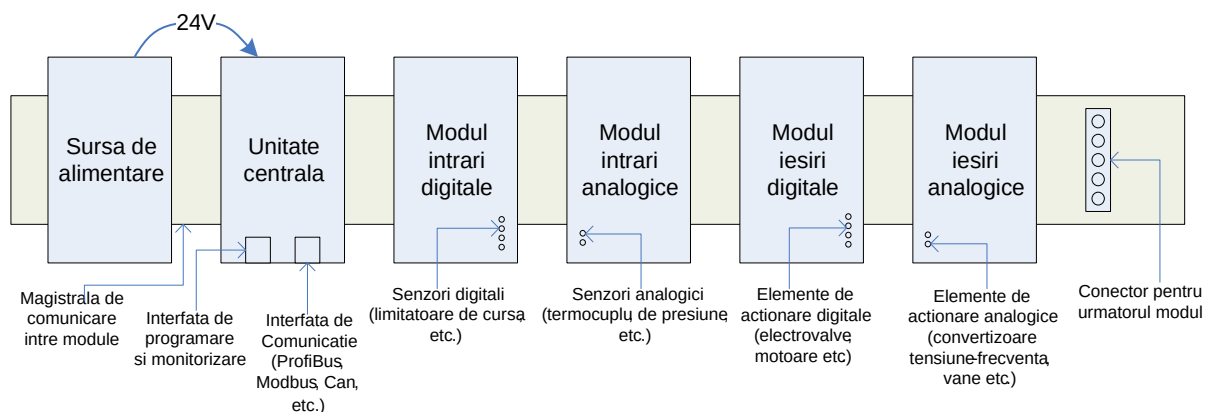


Fig. 1.3. – structura modulară a unui automat programabil

Exemplul din figura 1.3. conține structura fizică a unui automat programabil modular. Componentele acestuia sunt:

- Sursa de alimentare – oferă tensiunea tipică de lucru necesară (24 volți) pentru a alimenta Unitatea Centrală.
- Magistrala de comunicare este folosită pentru a conecta modulele cu unitatea centrală.
- Unitatea Centrală conține procesorul automatului și memoria acestuia.
- Modul intrări digitale – conține circuite de interfațare necesare conectării senzorilor ce oferă informații booleene.

- Modul intrări analogice – conține circuite pentru a interpreta tensiuni sau curenți oferți de senzori analogici.
- Modul de ieșiri digitale – conțin circuite prin care se pot comanda elemente de acționare cu comenzi booleene.
- Modul de ieșiri analogice – conțin circuite care pot genera tensiuni sau curenți de comandă.
- Conector pentru module adiționale.
- Interfața de programare și monitorizare necesară introducerii programului utilizator și monitorizarea acestor comenzi.
- Interfața de comunicare în mediu industrial prin protocol standard cum ar fi: ProfiBus, Modbus, CAN, etc.

1.1.3.Ciclul automat.

Principala funcție a unui automat programabil este de a citi toate intrările modulelor sale și apoi să execute programul din memoria sa, care va determina oprirea sau pornirea utilajelor din instalație sau modificarea mărimii comenzilor. În realitate procesul de modificare a stărilor ieșirilor automatului se realizează în trei pași. Întâi se face citirea intrărilor și copierea datelor în registrul de memorie al intrărilor. Apoi, pe măsură ce procesorul execută programul, el activează sau dezactivează biți în memoria sa internă corespunzători ieșirilor sale (registrul imagine al ieșirilor). După ce procesorul a terminat execuția întregului program, se trece la actualizarea ieșirilor conform datelor din memorie, adică activarea/dezactivarea sau modificarea efectivă a ieșirilor.

Procesul de citire, procesare a programului și actualizare a ieșirilor se efectuează ciclic, imediat după actualizarea ieșirilor trecându-se la citirea intrărilor, de aceea se numește *ciclu automat* (figura 1.4.). Semnalul intern prin care se indică terminarea unui ciclu automat se numește EOS (end of scan).

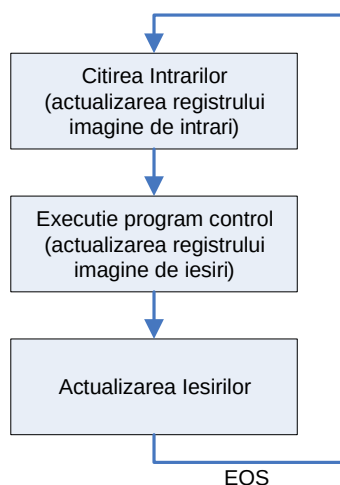


Fig. 1.4. – Ciclul automat

Fiecare din cei trei pași durează o anumită perioadă de timp. Timpul de răspuns al unui automat reprezintă durata totală a unui ciclu automat și constituie un parametru important al automatelor programabile (vezi figura 1.5.)[Bhowal,1999]. Timpul de execuție a programului depinde de doi factori:

- 1) Cantitatea de memorie ocupată de program

2) Tipul instrucțiunilor folosite în program (care afectează timpul necesar executării lor) Durata de timp necesară pentru un singur ciclu automat poate varia de la câteva zecimi de milisecundă până la 50 de milisecunde. Unii producători specifică timpul de răspuns ca funcție de cantitatea de memorie aplicație utilizată (de exemplu: 1milisecundă/1K de memorie program). Alți factori care pot influența timpul de răspuns pot fi legați de sistemul de intrări/ieșiri. Folosirea unor subsisteme I/O la distanță pot mări timpul necesar citirii intrărilor datorită necesității folosirii interfețelor de comunicare pentru a trimite/primi datele de la acestea.

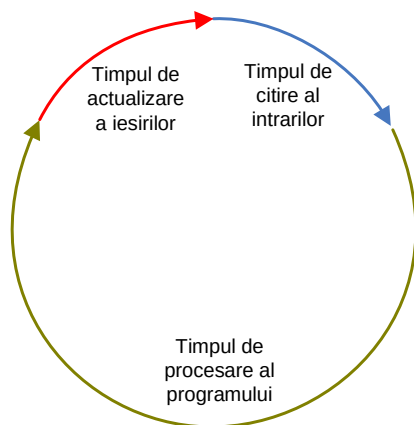


Fig. 1.5. – Timpul de răspuns al automatului programabil

Dacă se folosește funcția de monitorizare a automatului programabil de asemenea se va mări timpul de răspuns, deoarece procesorul trebuie să trimită datele interne către dispozitivul de monitorizare cum ar fi interfața cu utilizatorul.

Timpul de răspuns al automatului programabil influențează în mod direct posibilitatea acestuia de a detecta semnalele de intrare specifice. Un procesor este capabil să citească starea unei intrări atât timp cât semnalul nu este mai rapid decât timpul de răspuns (semnalul nu își schimbă starea de la “low” la “high” sau invers de două ori pe parcursul unui ciclu automat).

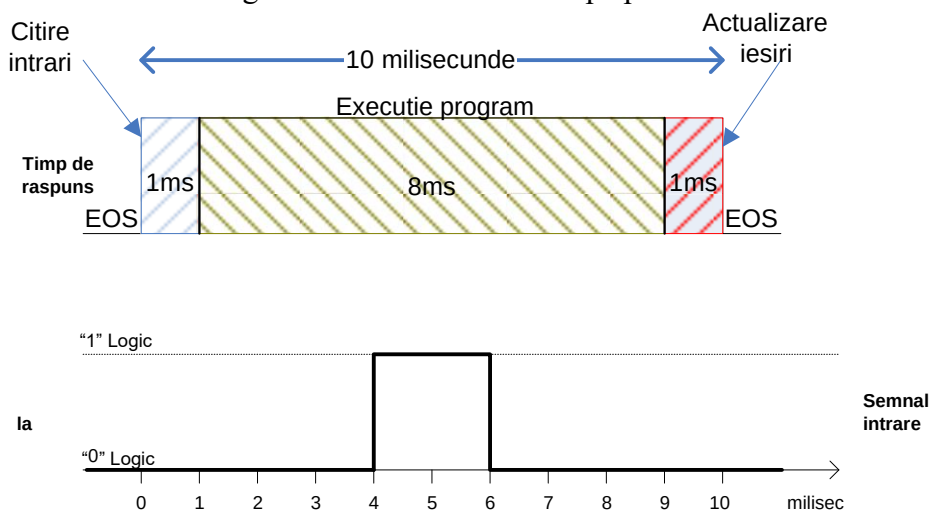
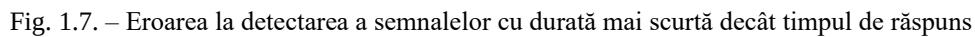


Fig. 1.6. – Eroare la detecție schimbare stare a semnalului de intrare.

Exemplul din figura 1.6. demonstrează necesitatea constrângerii anterioare. Aici timpul de răspuns al automatului este de 10ms, iar acesta trebuie să monitorizeze un semnal de intrare care își schimbă starea de două ori într-un interval de 8ms (mai mic decât durata ciclului). În

Dacă totuși semnalul apare în perioada de citire a intrărilor atunci el este detectat (a se vedea figura 1.7.), însă următorul nu va fi luat în considerare pentru că va apărea în perioada de actualizare a ieșirilor și când un nou ciclu începe semnalul deja are starea “low”.



The diagram illustrates the timing sequence for the 74LS147 decoder. The top part shows the output signals: Citire intrari (blue), Actualizare iesiri (red), and EOS (green). The bottom part shows the input signals: Semnal la intrare (blue) and Semnal prelungit (blue). The diagram illustrates the timing sequence for the decoder, showing the duration of the input signal and the time taken for the output signals to change.

Fig. 1.8. – Detecția semnalelor de durată mică prin prelungirea semnalului. Soluția este de a folosi un modul cu răspuns rapid pentru citirea acestui tip de semnal. Acesta conține un circuit care realizează prelungirea semnalului (a se vedea figura 1.8.) pe durata a cel puțin un ciclu complet automat, în acest mod schimbarea de stare a semnalului va fi cu siguranță surprinsă de unitatea centrală în perioada de citire a semnalelor. Dacă totuși trebuie să fie citite mai multe pulsuri într-o perioadă scurtă de timp, atunci se poate utiliza un modul de numărare rapidă care să surprindă toate schimbările de stare ale semnalului și să trimită informația de tip numeric mai departe către procesor.

Citirea semnalelor de intrare rapide și aperiodice se poate face și prin întreruperea ciclului automat care să fie tratată la nivelul programului utilizator. Aceeași tehnică poate fi aplicată și semnalelor de ieșire a căror actualizare trebuie să se facă mai repede decât permite ciclul automat. Aceste semnale de obicei sunt considerate a avea o importanță critică pentru sistemul de comandă al procesului. Mecanismul prin care se realizează întreruperea este cel din figura 1.9.

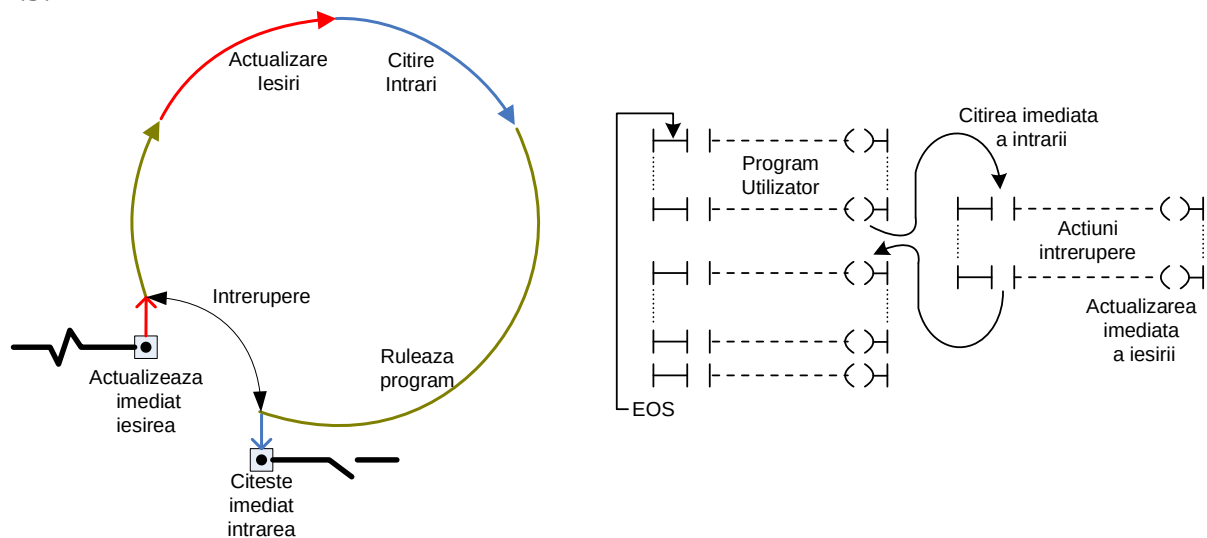


Fig. 1.9. – Mecanismul de tratare a semnalelor critice

În cazul în care semnalul apare în timpul execuției programului utilizator, procesorul este întrerupt din aceasta activitate și începe citirea imediată a intrării ce a generat întreruperea, efectuarea acțiunilor critice necesare și actualizarea imediată a ieșirilor critice. După terminarea acestor acțiuni se revine la rularea programului utilizator.

Majoritatea automatelor programabile au posibilitatea de a rula mai multe programe utilizator și de a le executa în paralel. Pentru aceasta se folosește un mecanism de multitasking care împarte ciclul automat, mai exact execuția programului utilizator, în felii de timp cu o anumită prioritate. Așa cum se poate observa cu ușurință din figura 1.10., primele executate sunt task-urile critice adică task-urile care implică elemente de intrare-ieșire critice.

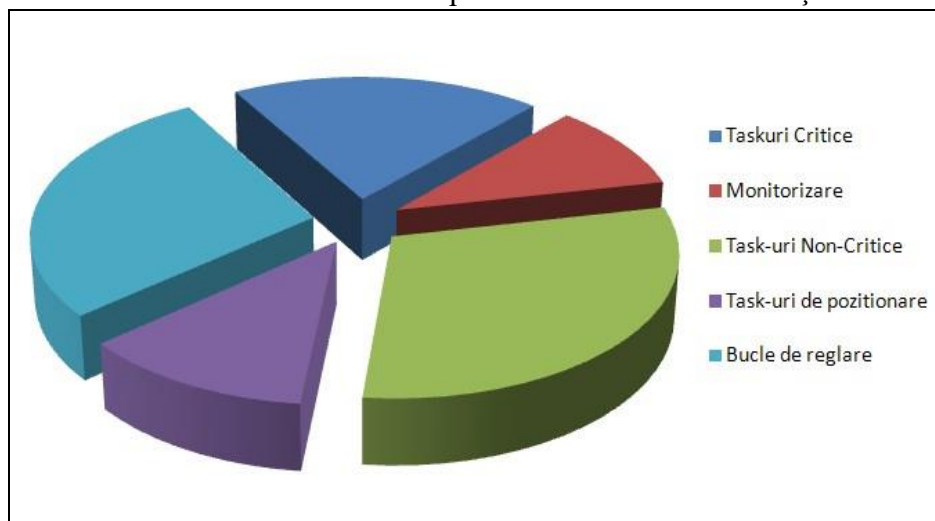


Fig. 1.10. – Structurarea task-urilor în ciclul automat

Următoarele executate sunt task-urile (eventual task-ul dacă este cazul) care ajută la monitorizarea sistemului și care trebuie să preia date din automat și să le transmită prin interfața de monitorizare către PC-ul sau dispozitivul cu care se realizează aceasta funcție. După încheierea execuției acestora urmează execuția task-urilor non critice. Deși task-urile de poziționare și buclele de reglare sunt de asemenea non-critice acestea au felii de timp separate.

1.2. Medii de programare pentru automatele programabile.

Producătorii de automate programabile de-a lungul timpului au propus diferite metode și limbaje pentru automatele proprii. „International Electrotechnical Commission” (IEC) a reușit să determine cinci limbaje comune care pot fi folosite atât pentru procese continue cât și pentru cele discrete. IEC este o organizație care pregătește și publică standardele internaționale privind toate tehnologiile electrice și electronice incluzând aici și automatele. Odată cu publicarea standardului IEC61131-3, organizația identifică cele cinci limbaje comune și abreviațiile lor: Ladder Diagram (LD), Instruction List (IL), Function Block Diagram (FBD), Structured Text (ST) și Sequential Flow Chart (SFC).

Program Organization Unit (POU) reprezintă cea mai mică unitate funcțională și independentă a unui program scris conform IEC61131-3 și poate fi o funcție, un bloc funcțional sau un program. Fiecare POU este constituit dintr-o zonă de declarații și un corp. Acest corp poate fi scris într-unul din limbajele de programare conform IEC descrise anterior. O funcție este un POU a cărui ieșire este un singur element de date (care poate fi format din mai multe elemente sau structuri). Funcțiile nu au elemente de stare internă, astfel că fiecare apel al unei funcții cu un anumit argument va avea același rezultat. La declararea unei funcții trebuie să se hotărască tipul de date returnat.

Un bloc funcțional este un POU a cărui rezultat poate fi diferit chiar dacă argumentul de intrare este același dar, spre deosebire de o funcție, blocul nu returnează o valoare. El poate avea mai multe instanțe (copii) ce au comportament particular în funcție de starea lor internă. Fiecare dintre instanțe are identificatorul propriu (numele instanței), și o structură de date ce conține intrările, ieșirile, și variabilele interne. Instanțele se declară local sau global ca variabile unde numele blocului este identificatorul.

Un program este un POU ce poate returna mai multe valori în timpul rulării. Programele sunt recunoscute global în proiect. Toate valorile se rețin de la ultima rulare până la următoarea rulare. Programele nu pot fi apelate de către o funcție, și nu pot exista mai multe instanțe ale aceluiași program. Dacă un POU a apelat un program, valorile interne ale acestuia modificându-se, acele valori sunt reținute și folosite la următoarea rulare chiar dacă programul este apelat și de un alt POU.

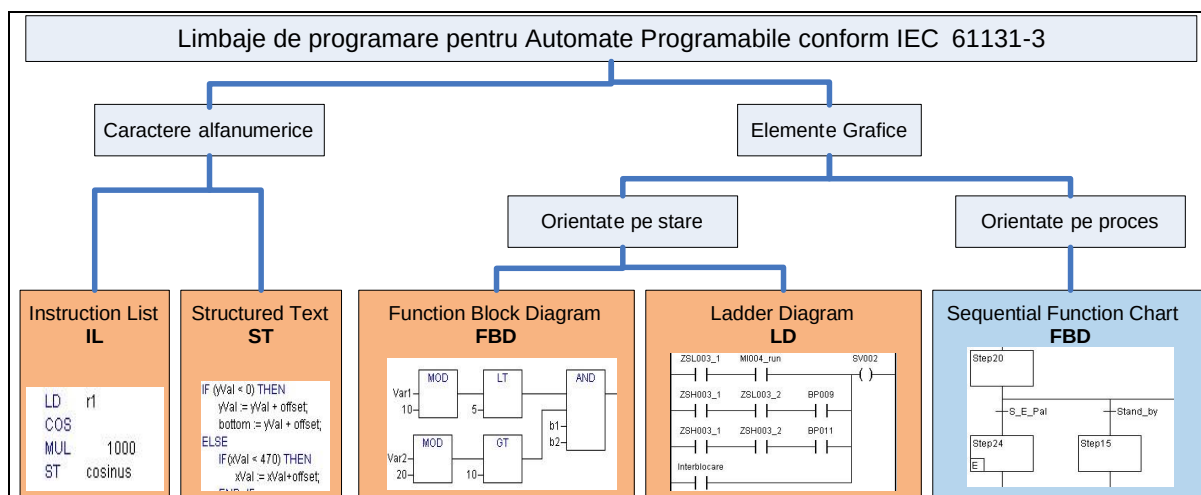


Fig. 1.11. – Clasificarea limbajelor în standardul IEC61131-3

Programarea conform standardului IEC 61131-3 reflectă deasemenea structura hardware și elementele de configurare. Aceste elemente sunt: configurații de bază – aici sunt definite elementele sistemului de comandă cum ar fi module de intrare/ieșire etc., resurse – unitatea centrală de procesare, variabile globale etc., și task-uri – care determină organizarea în timp a execuției programelor asociate lor. Task-urile pot fi de trei tipuri:

- Task-uri ciclice care sunt activate la anumite intervale de timp iar programul este executat periodic;
- Task-uri de eroare sunt activate la apariția unei probleme într-un alt task;
- Task-uri pe eveniment sau întrerupere care se activează la detectarea unui eveniment.

Deși inițial limbajele standard de programare ale IEC au fost folosite cu preponderență în Europa, acestea au început să fie folosite și de către producătorii din Statele Unite. Nevoia acestor standarde a fost simțită odată cu creșterea numărului de producători de produse de automatizare, complexității tot mai mare a aplicațiilor și metodele multiple de implementare a funcțiilor de control. Problema care se ridică este cea a utilității și criteriile după care se poate alege limbajul în care se va realiza aplicația. Pentru aceasta se va face o scurtă prezentare a celor cinci limbaje, clasificarea acestora fiind vizibilă în figura 1.11.

1.2.1.Limbajul de programare Ladder Diagram

Limbajul ladder diagram a fost inventat în Statele Unite ale Americii în anii 1970 și este probabil cel mai des utilizat pentru programarea automatelor. El a fost inventat pentru a înlocui sistemele de comandă cu relee fixate și momentan este folosit în aproximativ 95 de procente din totalul aplicațiilor. Vizual, acest limbaj este asemănător cu o serie de circuite, ale căror intrări trebuie să devină „adevărate” pentru a se activa una sau mai multe ieșiri.

Un program în Ladder Diagram este o înșiruire de ramuri. Întrucât instrucțiunile pot fi împărțite în două mari categorii, de intrare și de ieșire, pe orice ramură trebuie să existe cel puțin o instrucțiune de ieșire. Fiecare instrucțiune are asociat un simbol grafic și o valoare logică – TRUE sau FALSE.



Fig. 1.12. – Structura unei ramuri Ladder Diagram

În partea stângă se găsesc instrucțiuni de intrare (una sau mai multe) care formează întotdeauna o expresie logică de intrare. Expresia logică de la intrare este alcătuită din operatori logici (ȘI, SAU).

În partea dreaptă se găsesc instrucțiuni de ieșire, una sau mai multe. Efectul instrucțiunii de ieșire depinde de valoarea logică a expresiei de la intrare. Executarea ramurilor:

- se calculează expresia logică a instrucțiunii de intrare
- se execută instrucțiunea de ieșire în funcție de valoarea logică a expresiei de la intrare.

Programul este executat ciclic, deci se trece la scanarea primei ramuri după scanarea ultimei ramuri.

În cazul unui bloc AND se poate face reprezentarea grafică ca în figura 1.13, logica acestei ramuri fiind: Dacă A și B sunt adevărate atunci C devine sau rămâne adevărată.



Fig. 1.13. – Implementarea „AND”

În cazul unui bloc OR (sau logic) poate fi format, de exemplu, prin ramificarea instrucțiunilor ca în figura 1.14.



Fig. 1.14. – Implementarea „OR” Pentru a mări flexibilitatea realizării programelor, există posibilitatea de a introduce expresii logice complexe pe o ramură, mărind astfel complexitatea blocului logic generat și în mod asemănător poate avea loc și o ramificare a instrucțiunilor de ieșire. De asemenea se poate impune și o condiție suplimentară pentru emisia unei anumitor ieșiri (a se vedea figura 1.15.)

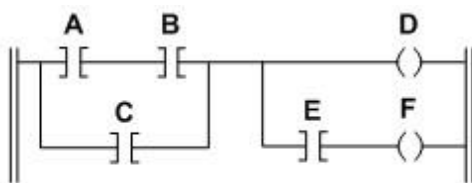


Fig. 1.15. – Condiționare suplimentară a unei ieșiri într-o ramură Ladder

În exemplul precedent, dacă (A AND B) AND E sau C AND E sunt adevărate, atunci F este adevărată.

Acest limbaj este relativ ușor de depanat în cazul în care nu funcționează conform așteptărilor doar pentru programe simple. Odată cu creșterea complexității programelor se simte nevoia dezvoltării de metode de diagnoză.

1.2.2.Limbajul de programare Instruction List Acest limbaj de programare este foarte ușor de înțeles și utilizat întru-cât se aseamănă foarte mult cu limbajul de asamblare. El constă din mai multe linii de cod fiecare dintre acestea reprezentând o singură instrucțiune/operație. Astfel

el are un format pas cu pas care facilitează folosirea diferitelor operații și instrucțiuni. Un alt avantaj este portabilitatea, în sensul în care, un program scris în limbajul Instruction List definit IEC poate rula pe mai multe platforme hardware independente care folosesc IEC 61131-3. Instruction List este un limbaj „low-level” și de aceea va rula pe automatul programabil mult mai repede decât limbajele grafice, cum ar fi Ladder Diagram sau Sequential Flow Chart. Un alt mare avantaj este faptul că este un limbaj compact și va utiliza mai puțină memorie și poate fi cu ușurință editat.

Deși are multe avantaje nu este foarte folosit de către inginerii care oferă servicii și mentenanță deoarece este mai greu de depanat și de înțeles. De asemenea la fel ca și în Ladder, funcțiile complexe matematice și programe bloc pentru controlul PID sunt foarte greu de realizat și introdus în programele Instruction List. Un alt dezavantaj este că nu poate fi integrat în oricare altă formă de programare structurată ceea ce îi micșorează aplicabilitatea. În plus, datorită creșterii vitezei procesoarelor utilizate în automatele programabile și a cantității de memorie disponibilă avantajul vitezei mare de rulare și codul compact devin din ce în ce mai mult irelevante.

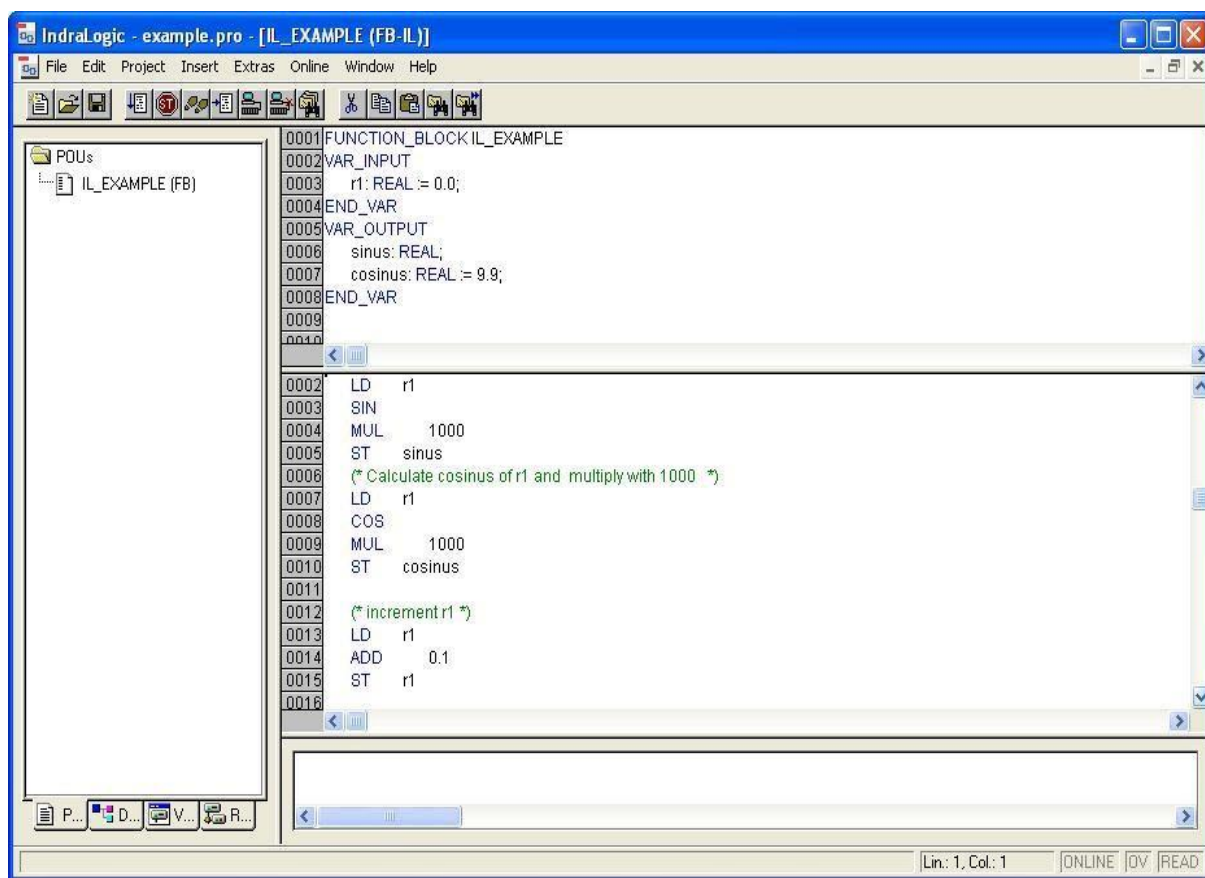


Fig. 1.16. – Exemplu de program în limbajul „Instruction List”

Un exemplu de program editat în limbajul „Instruction List” este cel din figura 1.16. în care se poate observa un calcul matematic. Datorită faptului că acest limbaj este foarte apropiat de cel de asamblare se poate face cu o oarecare ușurință conversia de cod de la limbajul Ladder Diagram la limbajul Instruction List.

1.2.3.Limbajul de programare Function Block Diagram Deși Ladder Diagram este cel mai răspândit în lume, se remarcă în ultima perioadă utilizarea tot mai deasă și a altor limbaje de

programare, de aceea Function Block Diagram este pe locul al doilea ca popularitate. Acesta se aseamănă cel mai bine cu o diagramă electrică (chiar mai mult decât Ladder Diagram), pentru că blocurile funcționale sunt interconectate în secvențe ușor de urmărit. Folosește aceleași instrucțiuni de bază ca și Ladder Diagram, dar din punct de vedere vizual este mult mai ușor de înțeles de către utilizator care nu este experimentat în logica releelor. Un alt avantaj este faptul că programele scrise în acest limbaj pot fi foarte ușor de urmărit și depanat. Limbajul este ideal pentru programe simple ce implică intrări digitale, cum ar fi senzori fotoelectronici, și ieșiri cum ar fi electrovalve etc. putând înlocui cu succes Ladder Diagram. Totuși nu este ideal pentru programe mari care folosesc funcții de intrări/ieșiri speciale.

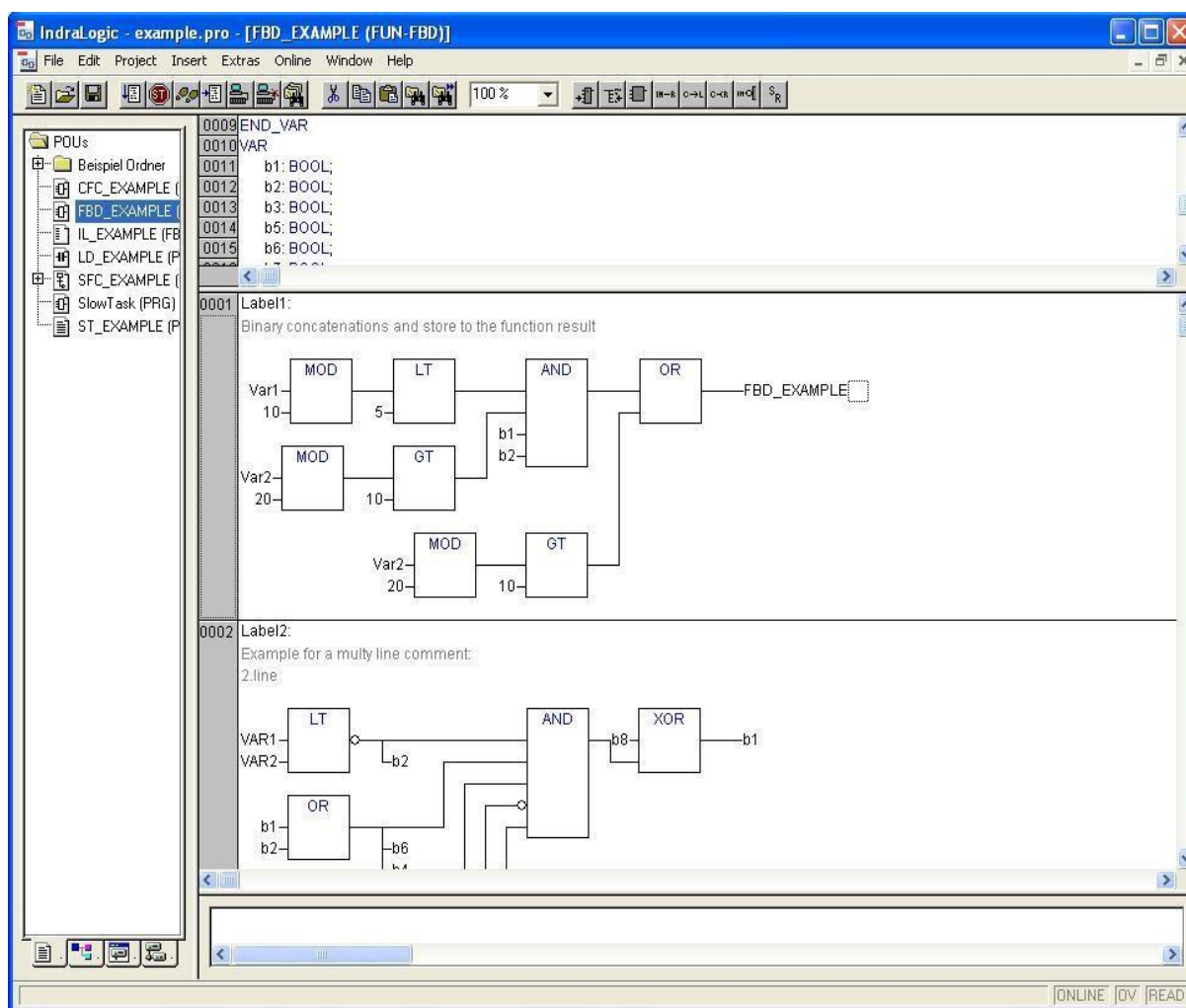


Fig. 1.17. – Exemplul de program editat în limbaj „Function Block Diagram”

Cantitatea mare de spațiu necesară de acest stil de programare poate face cu ușurință un program mai complex să devină greu de înțeles, ceea ce conduce la o depanare mai grea a programelor. Spre exemplificare se poate observa un program simplu în figura 1.17, unde se remarcă complexitatea redusă a programului față de suprafața de lucru relativ mare ocupată la editare. De asemenea scrierea de programe în Function Block Diagram necesită o bună preparare inițială a programului, iar utilizatorul trebuie să înțeleagă bine modul în care se execută sursa respectivă deoarece corecțiile ulterioare sunt mai dificile.

1.2.4.Limbajul Structure Text

Limbajul Structure Text este foarte asemănător cu limbajele de nivel înalt de programare cum ar fi PASCAL sau C, și are implementate bucle de tip WHILE, IF...THEN, CASE iar instrucțiunile se termină cu caracterul „;”. Acest limbaj se mulează cel mai bine pe tendința de evoluție a complexității programelor pentru automatele programabile. Funcțiile trigonometrice, analiza de date complexe, calculele matriceale pot fi realizate cu o mult mai mare ușurință decât în limbajele Ladder Diagram sau Instruction List.

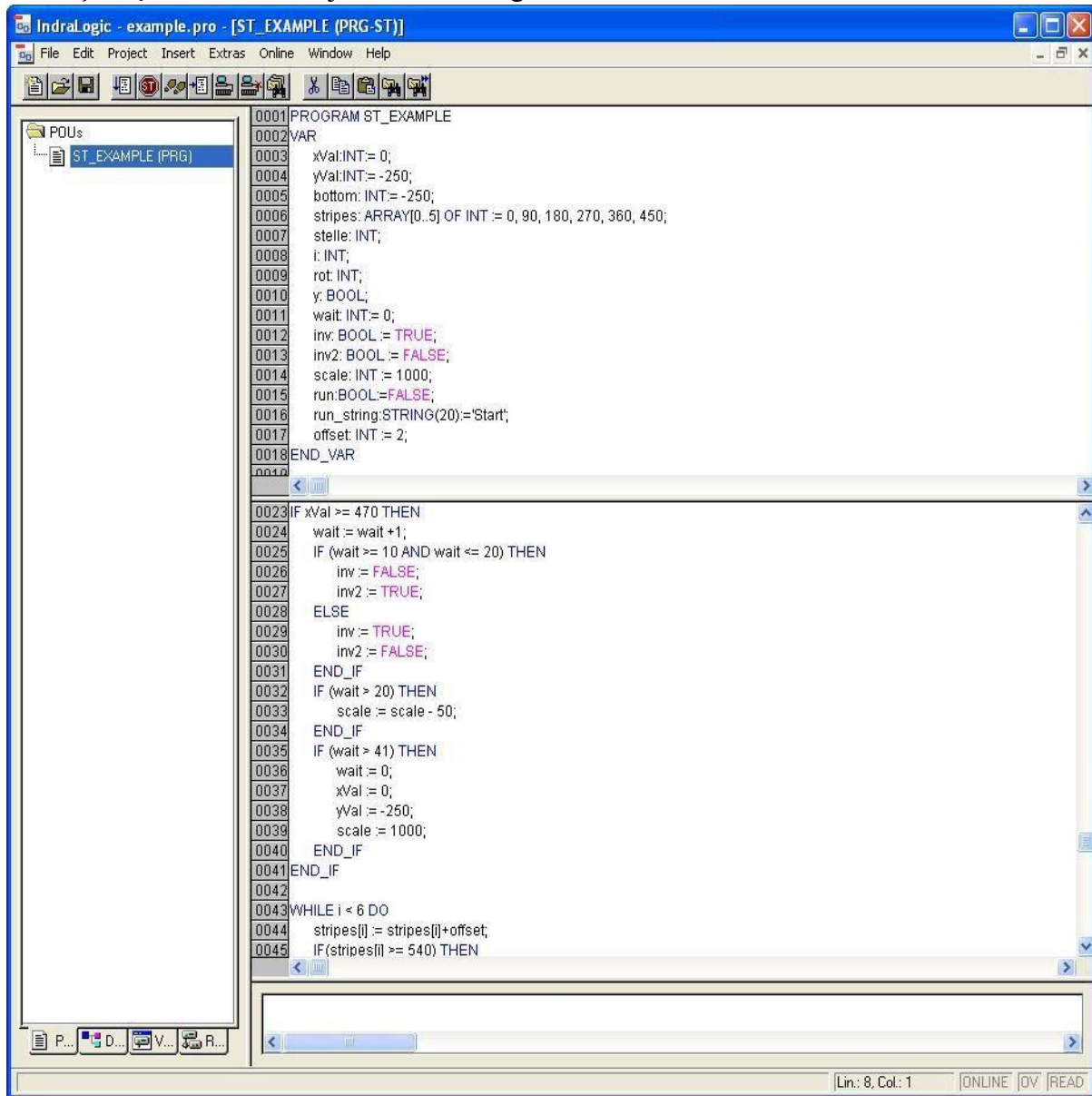


Fig. 1.18. – Exemplu de program în limbajul de programare „Structure Text”

Pentru a realiza bucle decizionale și a folosi pointeri (variabile folosite la adresarea indirectă) programul rezultat în Structure Text este mult mai compact decât cel în Ladder Diagram iar editorul oferă posibilitatea de a introduce comentarii pe parcursul codului pentru explicații ulterioare și cea de a mari indentari diferite pentru bucăți diferite de program. De asemenea, ca și Instruction List, rulează mult mai repede decât Ladder Diagram. Este de asemenea portabil, conform cu standardul IEC61131, putând fi folosit codul cu foarte mici modificări pe orice platformă hardware care suportă acest standard, ceea ce face ca programele

dezvoltate pe un automat programabil să poată fi reutilizate și pentru alte asemenea dispozitive. Un alt avantaj major este faptul că funcțiile definite în Structured Text pot fi încapsulate și utilizate în alte limbaje de programare. Un exemplu de program poate fi observat în figura 1.18.

1.2.5.Limbajul Sequential Function Chart

Sequential Function Chart (SFC) este un limbaj grafic utilizat pentru a descrie operații secvențiale. Procesul este reprezentat printr-o succesiune de pași finiți expliți legați prin tranziții a se vedea figura 1.19. Fiecărei tranziții îi este atașată o condiție de tip boolean. Acțiunile ce au loc la fiecare pas sunt descrise folosind celelalte limbaje (ST, IL, LD sau FBD). Un program SFC se reprezintă ca un graf orientat cu noduri de tip etape și tranziții. Legăturile multiple sunt utilizate pentru a reprezenta convergențe și divergențe. Regulile grafice de bază ale SFC:

- Un nod de tip etapă nu poate fi urmat de un nod de același tip
- Un nod de tip tranziție nu poate fi urmat de un nod de același tip.

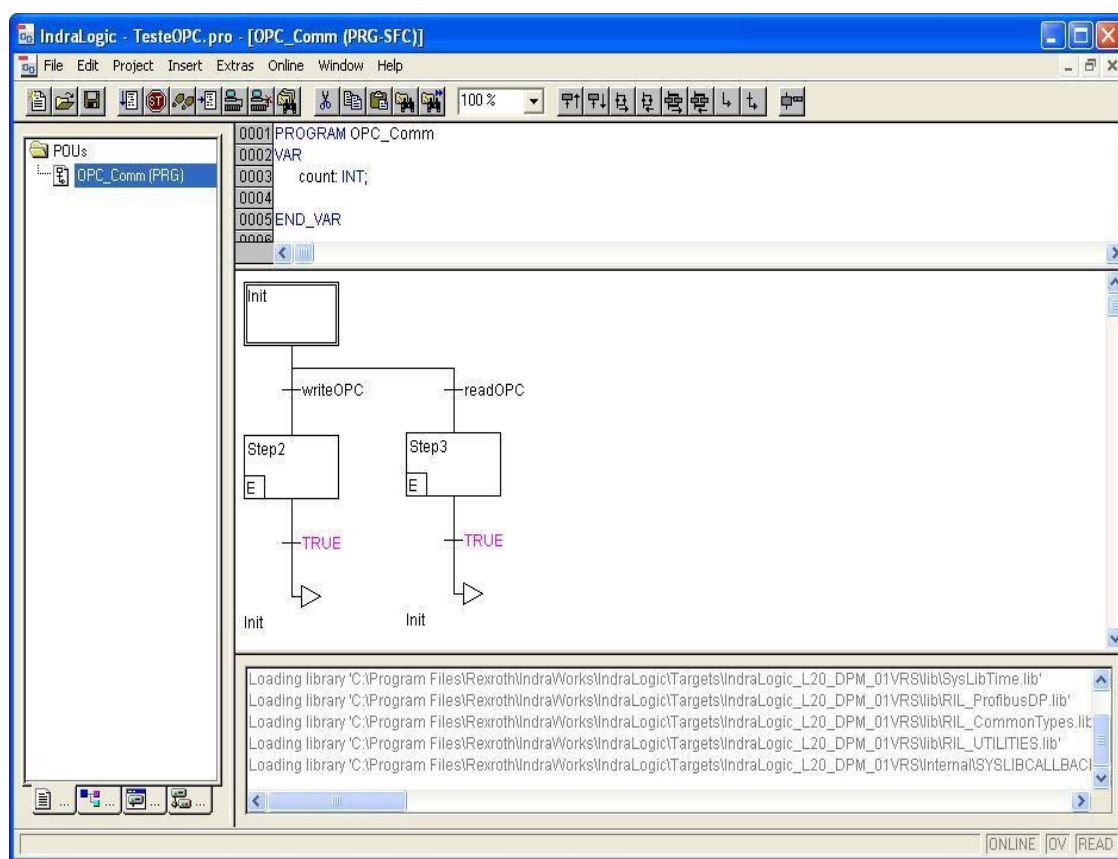


Fig. 1.19. – Exemplu de program în limbajul „Sequential Function Chart” Programarea SFC este de obicei separată în două niveluri :

- nivelul 1 arată organigrama programului, numerele pașilor și tranzițiilor și comentariile atașate acestora ca în figura 1.20.

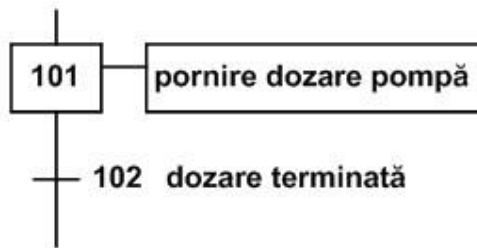


Fig. 1.20. – Nivelul 1 de organigramă a programului

- nivelul 2 reprezintă programarea în limbaj ST sau IL a acțiunilor din cadrul pașilor sau condițiile atașate tranzițiilor vizibil în figura 1.21.

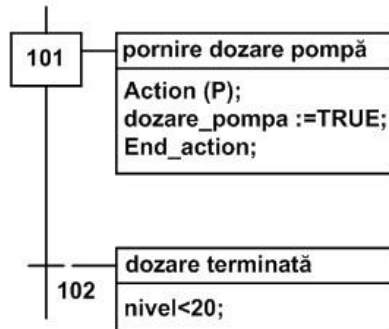


Fig. 1.21. – Nivelul doi de organizare a unui program SFC.

Componentele de bază (simbolurile grafice) ale limbajului SFC sunt:

- Etapele și etapele inițiale
- Tranzițiile
- Salturile către etape
- Divergențele
- Paralelisme

- **Etapă (pas)** (step): corespunde unei stări stabile a sistemului; în general au acțiuni atașate

O etapă este reprezentată printr-un pătrat.

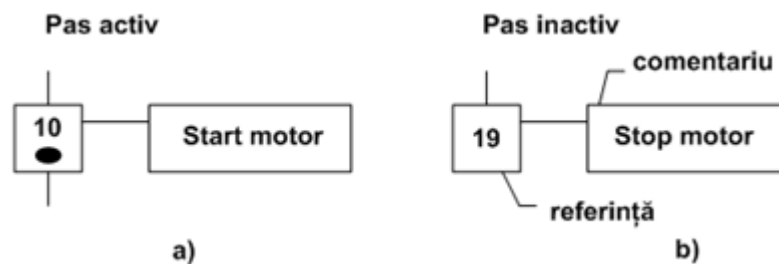


Fig.1.22. – Exemplu de pași

Starea inițială a unui program SFC se exprimă cu ajutorul etapelor inițiale, reprezentate grafic printr-un pătrat cu margine dublă. Când programul este pornit automat se plasează câte un jeton în interiorul fiecărei etape inițiale.

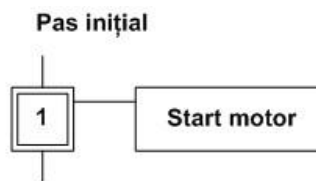


Fig.1.23. – Exemplu de pas inițial Un program SFC trebuie să conțină cel puțin o etapă inițială. Fiecare etapă are două atribute ce pot fi utilizate de celelalte limbaje:

- GSnnn.x starea pasului: activ/inactiv (variabilă booleană)
- GSnnn.t cât timp este activ pasul (timp) unde nnn este numărul de referință al pasului.

- **Tranziția** (transition): reprezintă tranziția între stări stabile ale sistemului. Fiecare tranziție are asociată o condiție de tranziție care este o expresie de tip boolean (poate fi și TRUE în permanență).

Tranzițiile sunt reprezentate printr-o bară orizontală ce taie linia de legătură dintre doi pași. Similar etapelor, tranzițiile sunt referite printr-un număr și au aceeași organizare pe nivele:

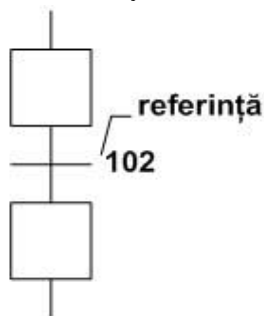


Fig.1.24. Exemplu de tranziție

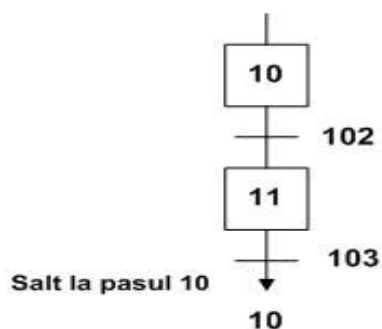


Fig. 1.25.- Exemplu de salt către o etapă

- **Salt către un pas** (jump to a step): indica o legătura dintre o tranziție și un pas, fără a fi nevoie să se deseneze linia de conexiune

Simbolul de salt poate fi utilizat pentru a indica o legătura dintre o tranziție și un pas, fără a fi nevoie să se deseneze linia de conexiune. Saltul trebuie referit cu numărul pasului destinație. Saltul nu poate fi utilizat pentru a lega un pas și o tranziție.

Divergențele și convergențele sunt utilizate pentru a reprezenta legături multiple între pași și tranziții.

- **Divergențele** sunt conexiuni multiple de la un simbol SFC (pas sau tranziție) către mai multe simboluri SFC. Convergențele sunt conexiuni multiple de la mai multe simboluri SFC către un singur simbol. Structurile de acest tip pot fi simple sau duble după cum urmează:
- Divergențe simple: Legături multiple de la un pas la mai multe tranziții. Aceasta permite trecerea jetonului activ către mai multe ramuri. Divergențele simple reprezintă posibilitatea rulării uneia dintre părțile unui proces (funcția OR):
- Convergențe simple: Legături multiple de la mai multe tranziții către același pas. Sunt folosite în general pentru a grupa ramuri ce s-au despărțit de la aceeași tranziție.

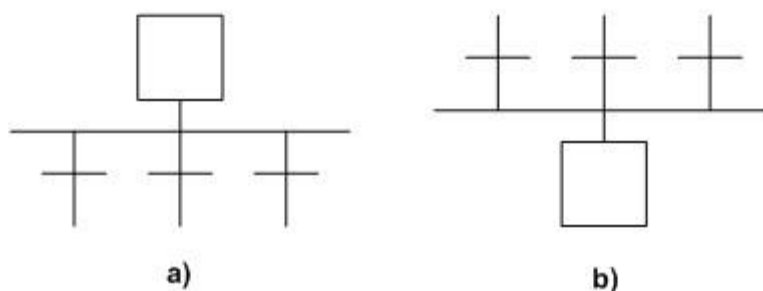


Fig. 1.26. a) divergență simplă; b) convergență simplă

- **Divergențe dublă:** Legături multiple de la o tranziție către mai mulți pași. Au corespondențe în execuția în paralel a operațiunilor într-un proces.
- Convergențe duble: Legături multiple de la mai mulți pași către aceeași tranziție. Folosite în general pentru a grupa ramuri ce s-au despărțit prin aceeași dublă divergență.

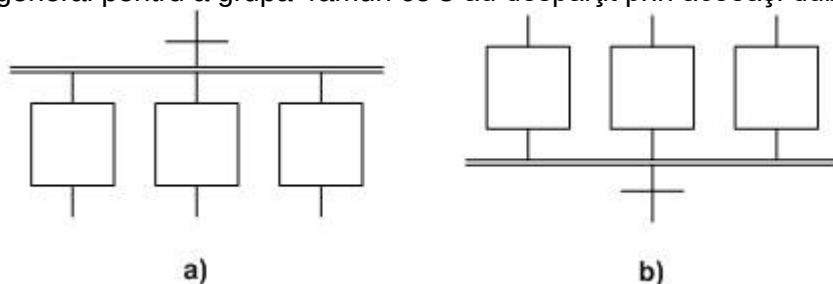


Fig. 1.27. a)divergență dublă; b) convergență dublă

Macro-etapele sunt un mod unic de reprezentare a unei succesiuni unice de pași și tranziții. Corpul unui macro este descris separat în programul SFC, el apărând ca un singur simbol în reprezentarea SFC principală. Numărul de referință ce apare în simbolul macro-ului este referința pentru primul pas din corpul unui macro. Acest prim pas trebuie să fie de tipul pas de început (beginning step). Corespondentul său la terminarea reprezentării este pasul de sfârșit (ending step).

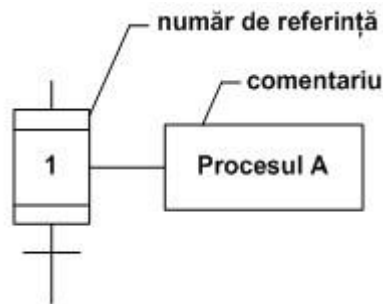
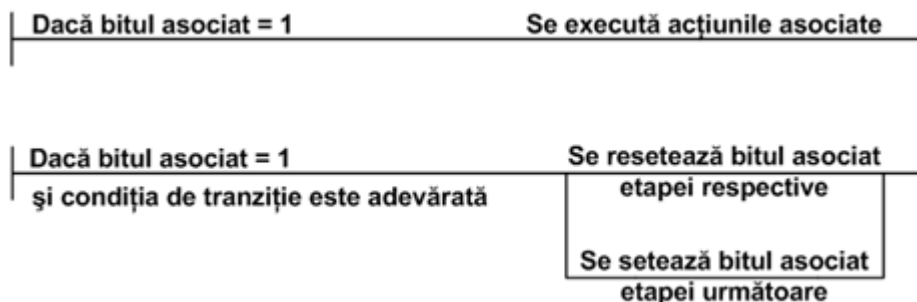


Fig. 1.28. Exemplu de macro-etapă

Reprezentarea unui macro trebuie să fie independentă, neexistând legături în nici o direcție. Simbolul unui macro poate fi inclus în reprezentarea altui macro. Deoarece un macro este reprezentat printr-un set unic de pași și tranziții, nu poate fi folosit mai mult decât o dată în programul SFC.

1.2 Dezvoltarea unui program Ladder pornind de la o diagramă Grafcet (SFC)

- Pentru ca un program de control conceput sub forma unei diagrame Ladder să funcționeze corect și să fie dezvoltat ușor, o soluție foarte eficientă este ridicarea inițială a unei diagrame de tip SFC pornind de la specificațiile procesului
- Elementele principale ale unei diagrame SFC sunt etapele, tranzițiile, acțiunile asociate etapelor, divergențele, paralelismul etc. Etapele de creare ale unei diagrame Ladder sunt următoarele:
 - Se construiește diagrama SFC de control logic al procesului
 - Se identifică intrările, ieșirile necesare automatului, precum și numărul de temporizări, contorizări necesar
 - Se asociază intrările și ieșirile cu biți din fișierele de imagine ale intrărilor și ieșirilor
 - Se asociază fiecare etapă a diagramei SFC cu câte un bit dintr-un fișier de tip BIT al automatului
 - Se alocă pentru fiecare temporizare sau contorizare câte un element dintr-un fișier de tip Timer sau Counter.
- Pentru fiecare etapă se scriu, în principiu două ramuri de tipul:



- Mici modificări apar atunci când în diagrama SFC există divergențe și paralelisme.

- În cazul în care dintr-o etapă se poate tranzita în diferite alte etape din cauza existenței mai multor condiții de tranziție, trebuie editată câte o ramură de tip 2 pentru fiecare condiție de tranziție. Starea de activare a unei etape se testează cu o instrucțiune de test aplicată bitului asociat etapei.
- Dacă ulterior activării unei tranziții urmează un paralelism în care mai multe secvențe vor rula simultan, o dată cu dezactivarea etapei curente vor fi activate toate etapele inițiale ale tuturor secvențelor paralele.
- Ieșirile digitale pot fi activate și dezactivate folosind instrucțiunile de setare/resetare a biților imagine a ieșirilor.

2. Module de intrare-ieșire pentru automate programabile.

În general sunt două tipuri constructive de module de intrare folosite în industrie: „sourcing” și „sinking” denumire care se referă la modul în care curentul electric intră sau iese din modulul respectiv.

Curentul electric are sensul de la terminalul pozitiv la cel negativ. Dacă modulul este de tipul „sourcing” așa cum se poate observa și în figura 1, curentul parcurge circuitul din terminalul intrării, acesta fiind un terminal pozitiv, prin senzor către masa comună a modulului de intrare. Cât timp senzorul nu închide circuitul, intrarea pe care senzorul este legat nu va fi activă. Când senzorul este activ, acesta va închide circuitul, ceea ce va duce la activarea intrării automatului.

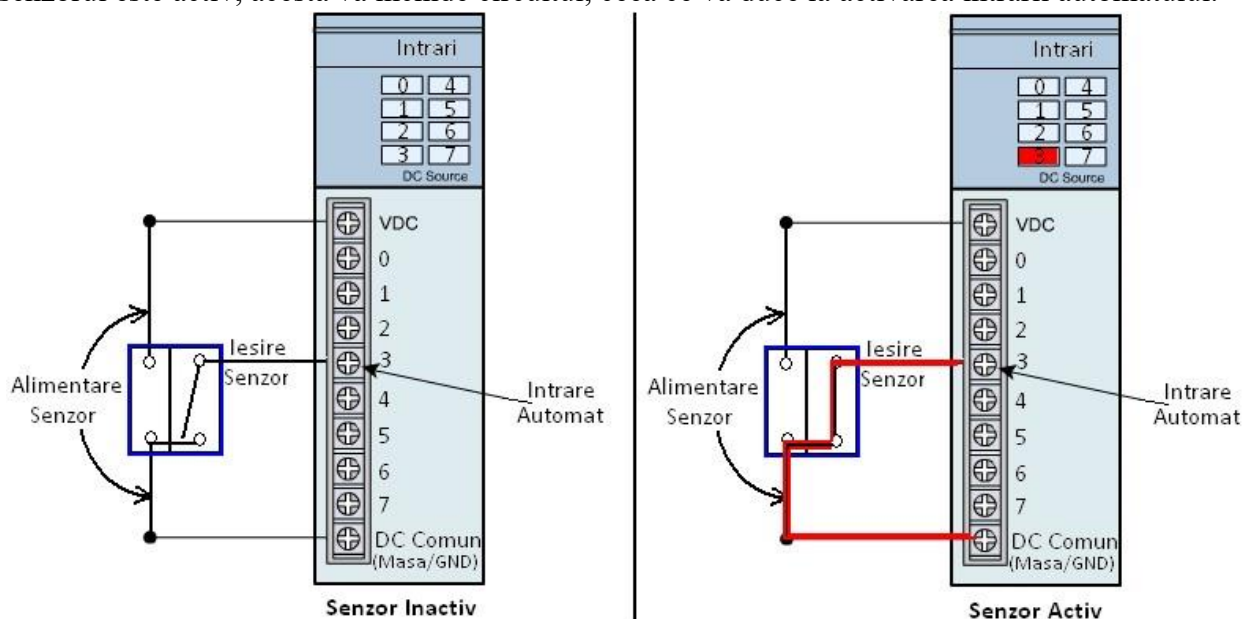


Figura 2.1 – Modul de intrare de tip “sourcing”

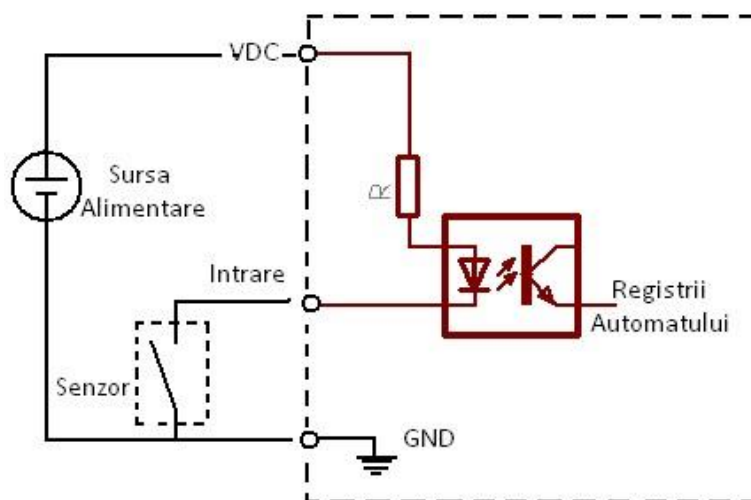


Figura 2.2 – Schemă electronică internă a modului tip ”sourcing”

Schema internă de funcționare a modului de intrare este cea din figura 2. În interiorul cardului se află un optocuplor pentru a se realiza izolația galvanică între circuitul industrial și cel al unității centrale. Dacă senzorul închide circuitul, va aduce potențialul de 0V al sursei de alimentare pe terminalul modului de intrare, făcând posibilă apariția curentului necesar pentru aprinderea ledului optocuplorului, ceea ce va duce în final la setarea registrelor de intrare a automatului.

Dacă modulul este de tipul „sinking” așa cum se poate observa în figura 3, curentul parcurge circuitul din terminalul pozitiv al sursei prin senzor către terminalul modului de intrare (în acest caz 5). Cât timp senzorul nu închide circuitul, intrarea pe care senzorul este legat nu va fi activă. Când senzorul este activ, acesta va închide circuitul, ceea ce va duce la activarea intrării automatului.

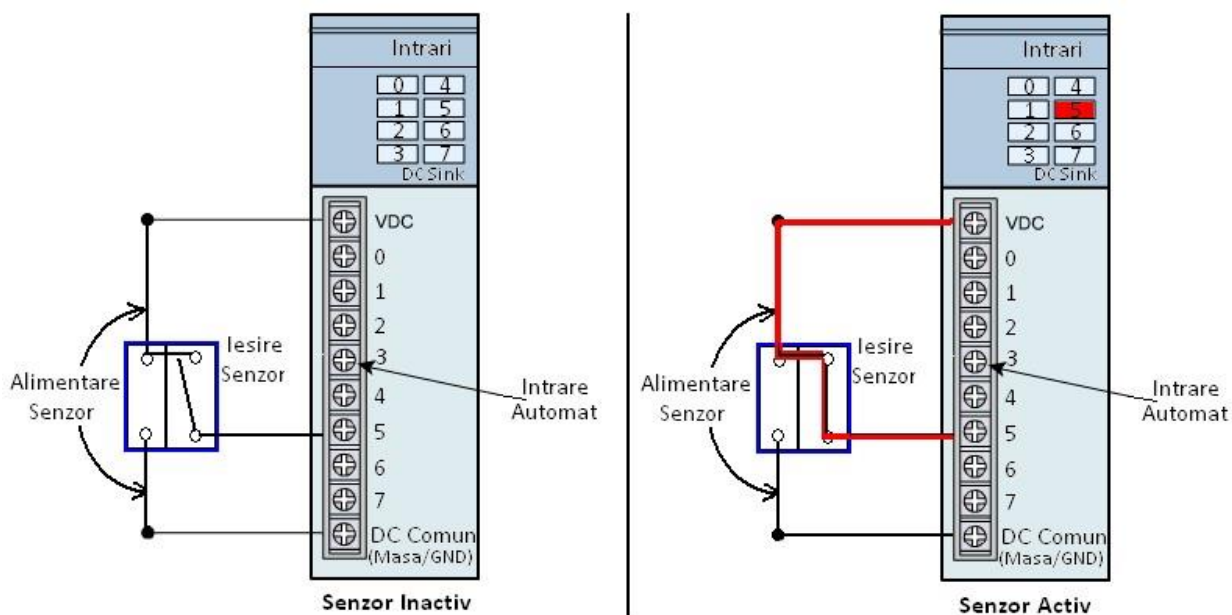


Figura 2.3 – Modul de intrare tip „sinking”

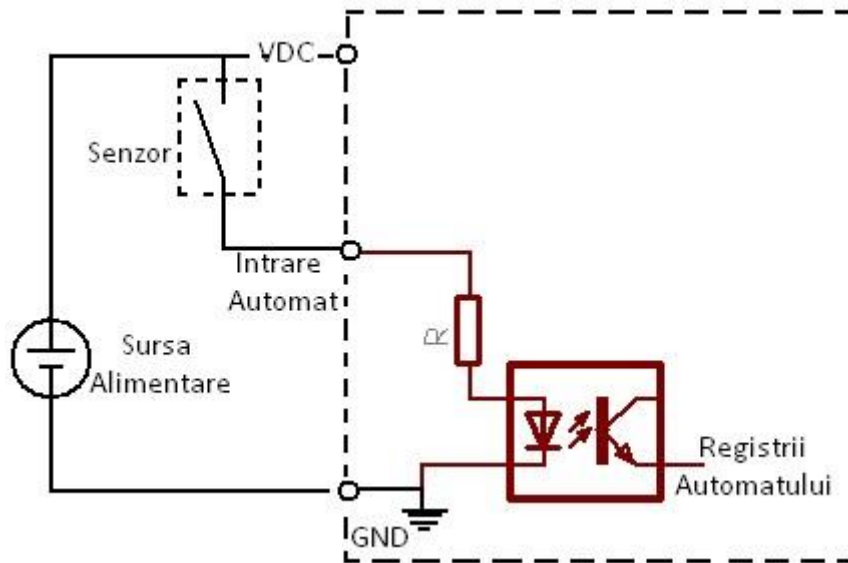


Figura 2.4 – Schema electronică internă a modului tip „sinking”

Schema internă de funcționare a modului de intrare este cea din figura 4. În interiorul cardului se află optocuplorul conectat între terminalul de intrare și masa comună. Dacă senzorul închide circuitul, sursa de alimentare va debita curentul necesar pentru aprinderea ledului optocuplorului prin intermediul contactului senzorului și a rezistenței R, ceea ce va duce la setarea registrului de intrare a automatului.

Module de ieșire.

Modulele de ieșire pot fi și ele de mai multe tipuri. Din punct de vedere al modului în care curentul electric iese sau intră în terminal pot fi de tipul „source” sau „sink”. În figura 2.5 este prezentat un modul de tip „sink”. Se poate observa că modulul conectează potențialul de 0 volți la terminalul său atunci când regiștrii automatului setează ieșirea respectivă. Când modulul este inactiv, la ieșirea acestuia există un potențial ridicat, circuitul fiind deschis. Când ieșirea a fost activată, tranzistorul intră în regimul de saturație comportându-se ca un contact, ceea ce duce la închiderea circuitului.

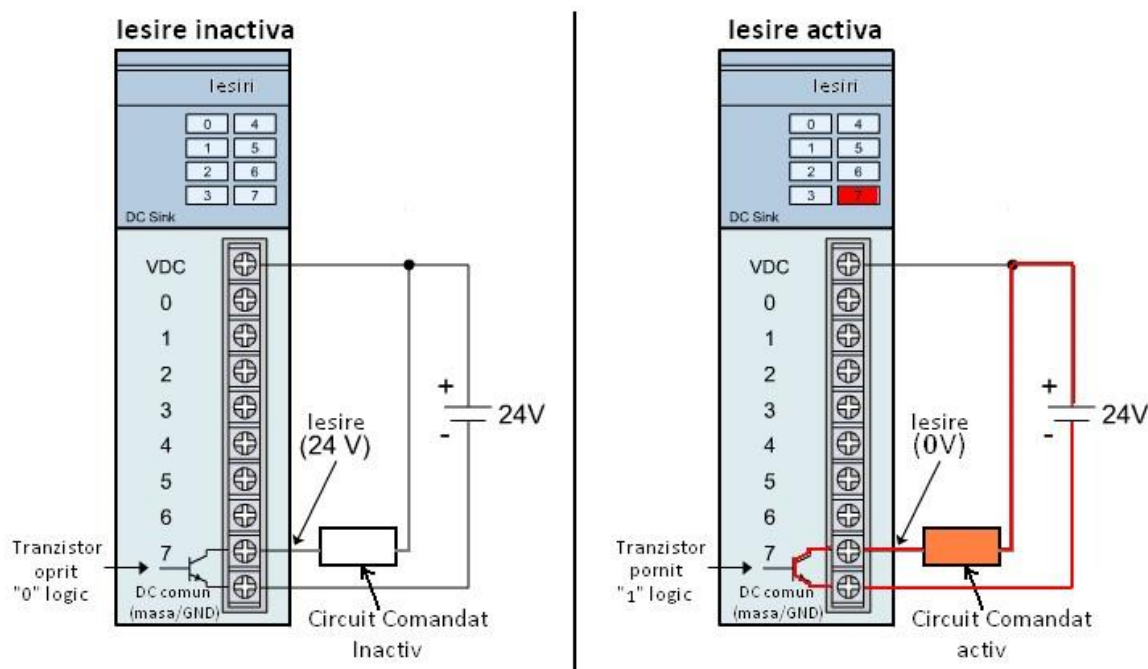


Figura 2.5 – Module de ieșire cu tranzistori tip „sink”

Modulele de ieșire tip „source” sunt reprezentate în figura 2.6. Se poate observa că spre deosebire de tipul anterior de module, acestea vor conecta potențialul pozitiv de 24 Volți al sursei de alimentare la circuitul comandat. În momentul când ieșirea este inactivă, tranzistorul corespunzător ieșirii respective este blocat, ceea ce înseamnă pentru circuitul comandat absența unui potențial pozitiv la borna sa. Când din regiștrii automatului are loc activarea (setarea) acestei ieșiri, tranzistorul intră în saturație, acționând ca un simplu contact. Atunci se închide circuitul ceea ce conduce la apariția potențialului pozitiv la borna circuitului comandat, acesta fiind astfel activat.

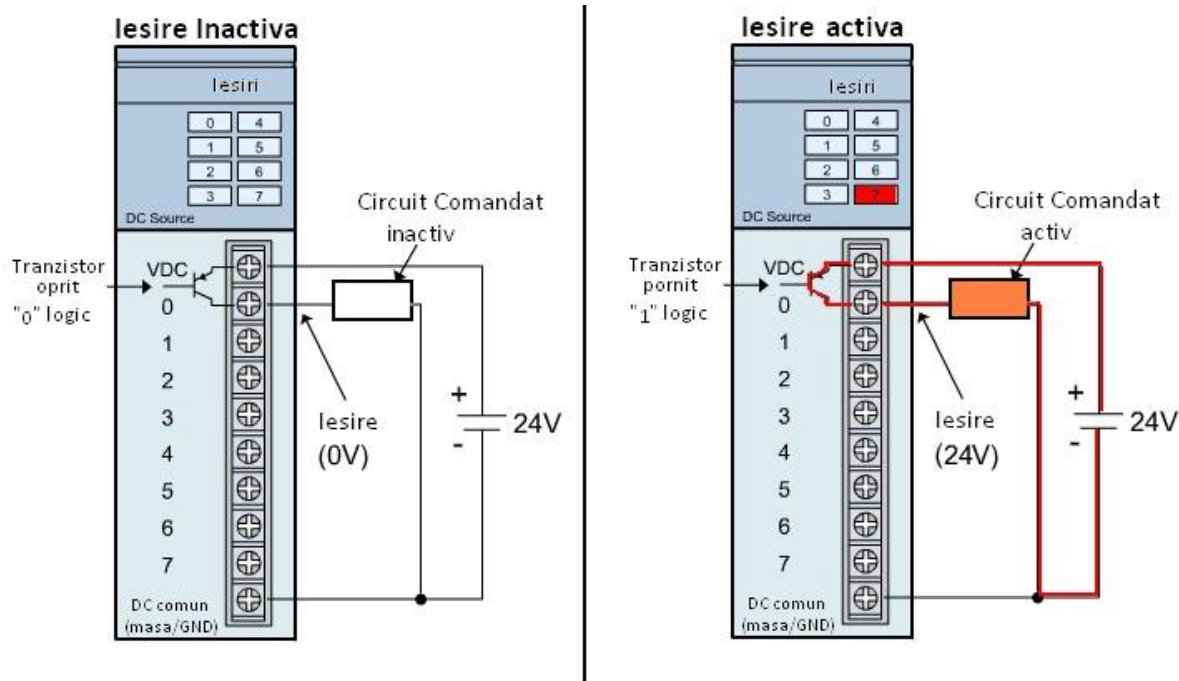


Figura 2.6 – Modul de ieșire cu tranzistori tip „source”.

Dezavantajul major al modulelor cu tranzistoare este limitarea de curent și tensiune impuse de acestea. Cele mai multe astfel de module nu oferă posibilitatea activării de circuite comandate la o tensiune mai mare de 24 de Volți curent continuu și un consum mai mare de 500mA. De aceea marii producători de automate programabile dau posibilitatea echipării cu module de ieșire cu rele. În figura 2.7 se poate vedea un modul cu rele tip „source”

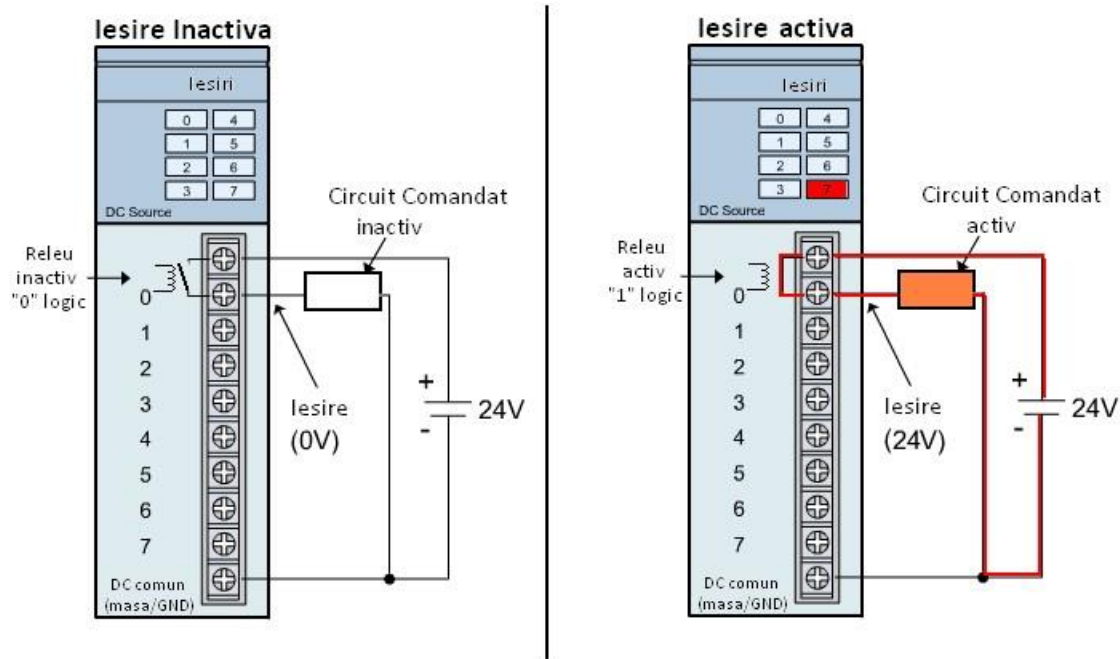


Figura 2.7 – Modul de ieșire tip „source” cu rele.

Contactele releelor pot suporta în general tensiuni de până la 240 Volți, iar un alt mare avantaj este faptul că se poate folosi și curent alternativ. Curentul maxim admis de contactele releelor diferă de la producător la producător dar ca un minim general s-a ales valoarea de 500mA la tensiunea de 240V.

3. Tipuri de Control

Există mai multe tipuri de control:

- Control de Proces (Process Control): reprezintă o gamă de arhitecturi, mecanisme, algoritmi de menținere a ieșirilor unui proces într-o anumită gamă. Se împarte în:
- Control discret: Variabile care variază discret în timp. Folosit în aplicații de manufactură, mișcare, împachetare, asamblarea robotizată (cum ar fi de exemplu în industria auto)
- Control continuu: Deseori, un sistem fizic este reprezentat prin variabile continue și neîntrerupte în timp. Controlul temperaturii apei dintr-o încălzire este un exemplu. Procesele continue au loc în producția de carburanți, chimicale și plastic. Procesele continue sunt folosite pentru a produce cantități mari de produse finale anual.
- Control pe loturi (Batch Process Control): Unele aplicații necesită cantități specifice de materie primă combinate într-un anumit fel pentru anumite perioade de timp astfel încât să

producă un rezultat intermediar sau final. Exemple: producția de mâncare, băuturi, meciamente, adezivi etc.

- Control de Produs (Product Control): Product Automation reprezintă automatizarea etapelor individuale sau a etapelor întregului lanț de a produce ceva

Nivele ale controlului de proces:

1. Masură și comandă: < 1 secundă
2. Siguranță, protecție ambientală și a echipamentelor: < 1 secundă
3. Control regulator : secunde - minute
4. Control Multivariabil: minute - ore
5. Optimizarea în timp real: ore - zile
6. Planificare: zile – luni

Automatizarea de proces este folosită în scopul maximizării producției menținând un nivel dorit la calității produselor și al securității

Măsură și comandă:

- Dispozitive de măsură (senzori și traductoare) și echipamente de comandă (motoare, valve de control) sunt folosite pentru a măsura variabilele de proces și a implementa acțiunile de control necesare
- Dispozitivele sunt interfațate cu sistemul de control, de obicei un automat programabil
- Sunt indispensabile sistemului de control

Siguranță și protecție:

- Procesul trebuie să opereze în siguranță și să satisfacă condițiile de mediu
- Se bazează pe principiul straturilor multiple de protecție care implică combinația de echipamente și acțiuni umane
- Un strat include funcții de control al procesului, cum ar fi gestionarea alarmelor în cazuri de funcționare anormală și echipamente pentru oprirea de urgență (emergency shutdown)
- Echipamentele de siguranță operează independent de instrumentația obișnuită
- Tehnici de validare a stării senzorilor pot fi folosite pentru a confirma funcționarea lor corectă

Control Regulator:

- Variabile de proces ca: temperatură, presiune, flux etc trebuie să opereze cât mai aproape de valorile de referință
- Reglarea lor se obține prin tehnici standard de tip feedback și feedforward
- Dacă tehnicile standard nu satisfac, se folosesc tehnici mai avansate (fuzzy logic, rețele neuronale)

Controlul Multivariabil:

- Procese dificile:
 1. Interacțiuni semnificative între variabile importante din proces
 2. Constrângeri de inegalitate pentru variabilele controlate și manipulate
- Constrângerile includ limitele superioare și inferioare.
- Se folosesc tehnici de control avansate, spre exemplu MPC (model predictive control)

Optimizare în timp real:

- Condițiile de operare optimă se pot schimba frecvent datorită:
 - Disponibilitatea echipamentelor
 - Disturbări ale procesului
 - Condiții economice schimbate (ex. costurile materie primă)
- Condițiile de operare optimă trebuie recalculate regulat și stabilite noi valori de referință pentru variabilele controlate
- Acest nivel include și analiză de date pentru a asigura faptul că modelul procesului folosit în calculele RTO (Real Time Optimization) sunt corecte pentru condițiile actuale.
- Tehnici de reconciliere a datelor pot fi folosite pentru a asigura că balanțele energetice sunt satisfăcute (tehnologie care folosește informațiile din proces și metode matematice în scopul corectării automate a măsurătorilor din procesele industriale)
- De asemenea modelul procesului poate fi actualizat folosind tehnici de estimare a parametrilor și informații recente despre fabrică.

4. Arhitecturi hardware standard

Automatul programabil este folosit cu precădere în majoritatea aplicațiilor din domeniile de fabricație, distribuție sau transport materiale. Sistemele de tip PLC (Programmable Logic Controller) au apărut cu scopul inițial de a înlocui mașinile secvențiale cu relee destinate controlului aplicațiilor industriale (mașini unealtă, instalații industriale, etc.).

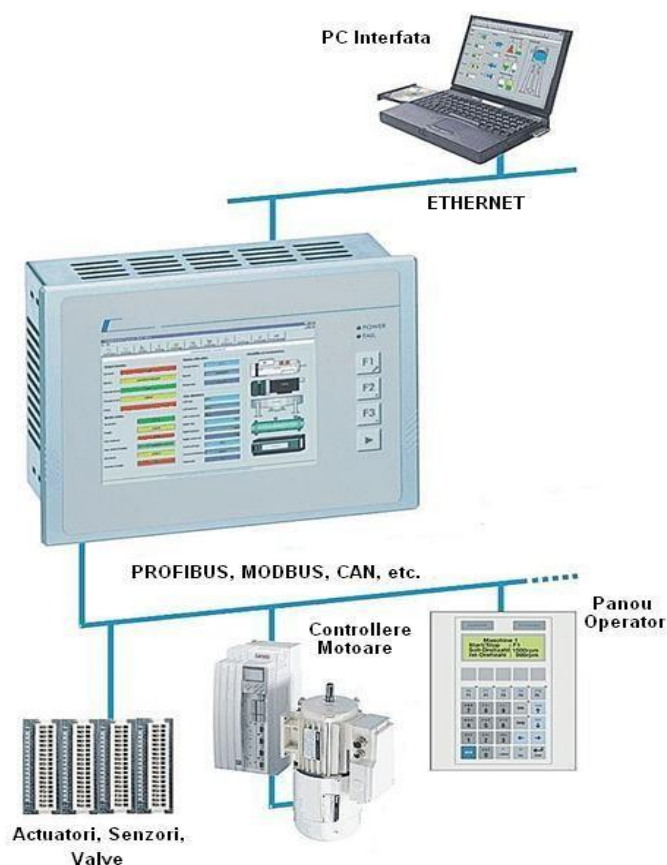


Fig. 4.1 – Arhitectura de bază cu Automat Programabil

Automatele programabile moderne pot susține nu numai nivelurile cele mai joase de automatizare dar de asemenea și sisteme hibride avansate de comanda, unde controllere în timp continuu trebuie să colaboreze cu cele cu evenimente discrete.

Din punct de vedere hardware topologiile de control cu automate programabile sunt dintre cele mai variate, fiind determinate de cerințele specifice fiecărei aplicații. Cea mai trivială astfel de arhitectură este prezentată în figura 4.1 și este specifică aplicațiilor simple care necesită pe lângă controlul elementelor din aplicația industrială (motoare, electrovalve, actuatore, etc.) și o interfață cu utilizatorul, fie printr-un panou local de comandă, fie prin acces la distanță cu ajutorul unui PC.

Această arhitectură hardware de bază este însă valabilă pentru procese cu un număr mic de componente de intrare – ieșire, deoarece depinde direct de performanțele automatului programabil. În general producătorii limitează numărul modulelor pe care un automat programabil le poate gestiona, pentru că timpul în care operațiile de scanare a intrărilor respectiv de actualizare a ieșirilor este direct proporțional cu acest număr și invers proporțional cu viteza de lucru ale unităților centrale de procesare.

Aplicațiile complexe cu sute de intrări-ieșiri au însă nevoie de arhitecturi hardware specifice în care sunt implicate mai multe automate programabile fiecare dintre acestea având în gestiune o zonă cu mai multe elemente de execuție. Automatele sunt interconectate între ele pentru a putea comunica parametrii elementelor din zona pe care o comandă, atât pentru a crea o imagine asupra evenimentelor procesului/proceselor din aplicație pentru utilizatorul uman, cât și pentru a putea facilita funcționarea corectă a zonelor adiacente implicate în proces.

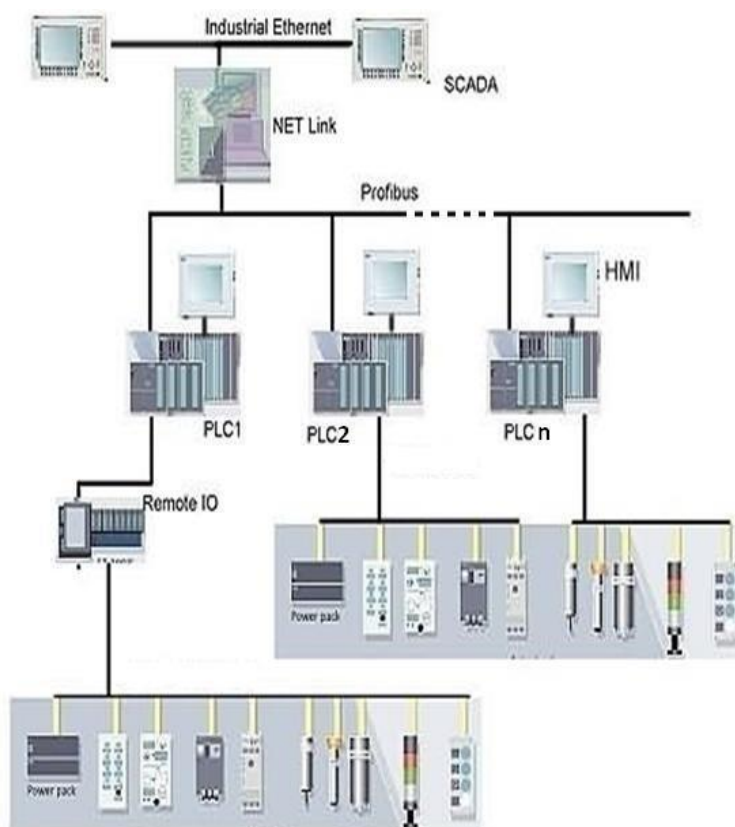


Fig. 4.2 – Arhitectură Complexă Cu Automate Programabile

În figura 4.2 se poate observa un exemplu de arhitectură complexă în care automatele programabile au același nivel de supervizare. Ele fie controlează o zonă simplă cum este cazul PLC 1, fie partajează împreună cu alte automate aceeași zonă de interes așa cum se întâmplă pentru PLC2 – PLCn, dar toate se află pe același nivel, realizându-se o cooperare fără subordonare, supervizarea făcându-se însă de către sistemul SCADA de control.

Alte arhitecturi presupun însă existența unor relații de tip master-slave între automate, așa cum se poate observa în figura 4.3, astfel că procesul este condus de un master ce dă comenzi către automatele slave, acestea gestionând elementele de execuție din zonă dar comenzile de execuție venind din partea automatului master.

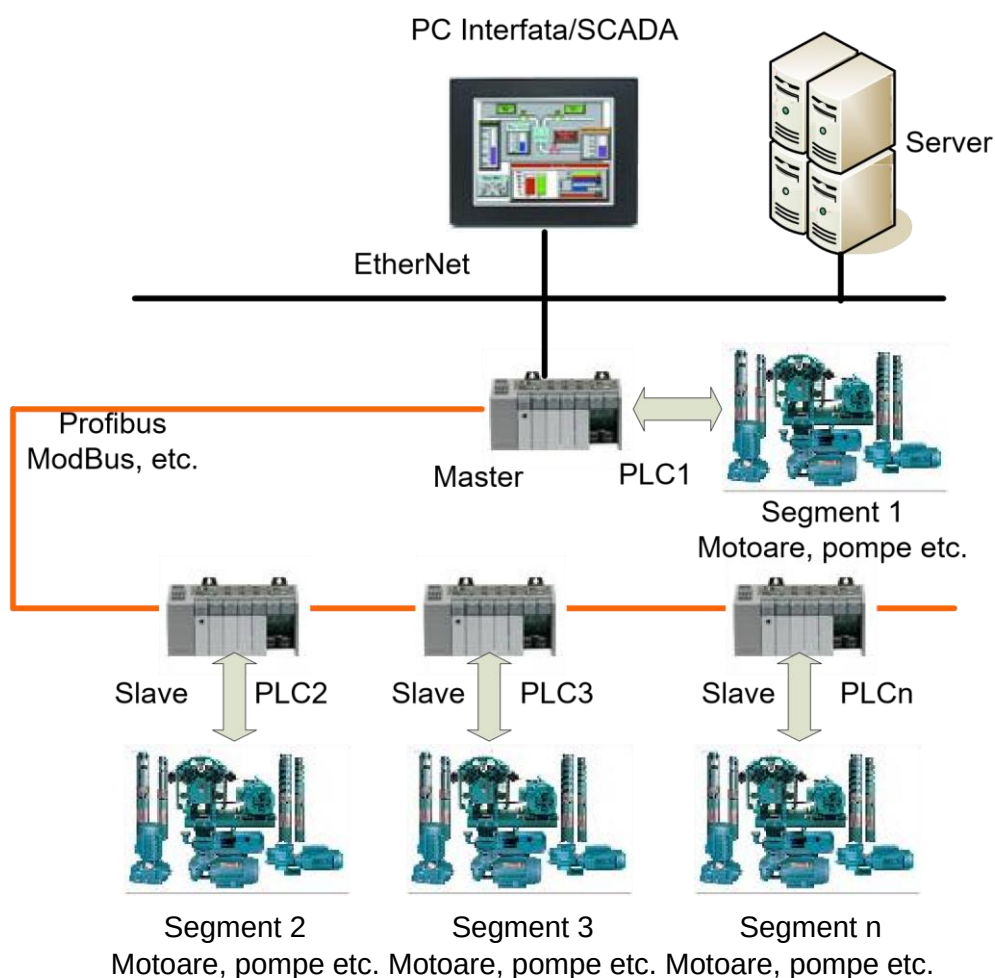


Fig. 4.3. – Arhitectura Master-Slave

Așa cum se vede din figura 2.3, un automat Master are drept rol negocierea parametrilor de funcționare cu o interfață de tip SCADA care se află pe un PC sau pe un panou operator dedicat, iar apoi interpretează aceste date și le organizează în seturi mai mici trimițându-le către automatele Slave. În același timp el preia de la acestea datele privind stările de funcționare și le recompune astfel încât operatorul uman să aibă imaginea de ansamblu asupra întregului proces industrial. Serverul poate găzdui și el aplicații care să comunice prin rețeaua EtherNet cu automatul Master, acesta din urmă servind datele necesare bunei funcționări a întregului sistem, fără a exista comenzi contradictorii.

5. Rețele de comunicație în mediul industrial: Profibus

PROFIBUS este o rețea de mare viteză pentru echipamente distribuite. Are trei versiuni compatibile:

- PROFIBUS DP (Decentralized Periphery), care este o rețea de mare viteză, de cost redus și cu un nivel de comunicație simplu (90% din rețele);
- PROFIBUS FMS (Field bus Message Specification), care constă în aplicații de nivel înalt de comunicație, utilizate în celule de control complex;
- PROFIBUS PA (Process Automation), dezvoltată pentru automatizarea proceselor, în special în zone periculoase.

DP și FMS au același mediu de transmisie (RS-485), în timp ce PA utilizează tehnologia MBA (Manchester coding and Bus Powered). Vitezele de transmisie pentru DP și FMS sunt: 9,6 Kbps, 19,2 Kbps, 93,75 Kbps, 187 kbps, 500 Kbps, 1,5 Mbps, 3 Mbps, 6 Mbps, 12 Mbps. Pentru PA viteza este 31,25 Kbps. O rețea este alcătuită din mai multe segmente, care se realizează utilizând repetoare. Fiecare segment trebuie să aibă la capete conectată o impedanță egală cu impedanța caracteristică a rețelei.

Protocolul de comunicație:

PROFIBUS utilizează un protocol bazat pe cadre de octeți. Cadrele sunt de lungime variabilă. Cei mai mulți utilizatori folosesc un chip special (SPC3), care are înglobat protocolul PROFIBUS. Comunicarea înseamnă transmitere și recepționarea de mesaje numite telegrame. Numai dispozitivul master poate iniția o comunicație, în timp ce dispozitivele slave răspund la solicitările masterului propriu.

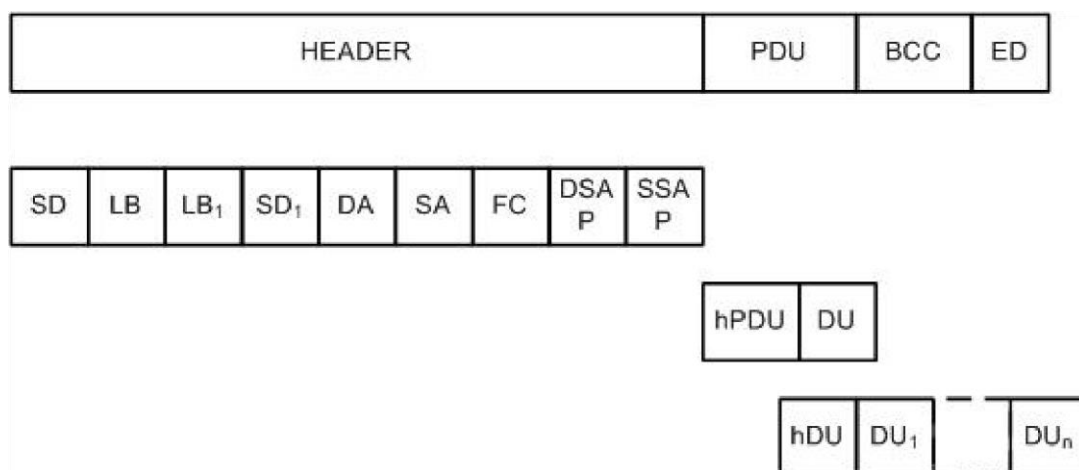


Fig. 5.1. – Structura cadrului de mesaje Profibus

SD și SD1 - delimitator de start; LB și LB1 - lungimea mesajului; DA - adresa de destinație; SA - adresa sursei; FC - codul funcției (tipul și prioritatea mesajului); DSAP - portul de comunicație al destinatarului; SDAP - portul de comunicație al expeditorului; PDU - unitatea de date (dacă există); BCC - blocul de control al erorii; ED - delimitator de sfârșit de mesaj (16H). Există și un tip de mesaj scurt din un singur octet E5H, care înseamnă acceptarea fără confirmare.

Profibus DP suportă implementarea a două tipuri de rețele: mono-master și multi-master. Componenta principală a unei rețele mono-master este un Automat Programabil. Dispozitivele slave sunt cuplate prin intermediul mediului de transmisie. Nodurile active din inelul logic au o ordine dată, care nu depinde de poziție. Dreptul de acces la rețea este legat de deținerea jetonului. Jetonul este pasat, după un timp unui alt nod activ, în ordinea logică. Nodurile pasive nu pot recepționa jetonul.

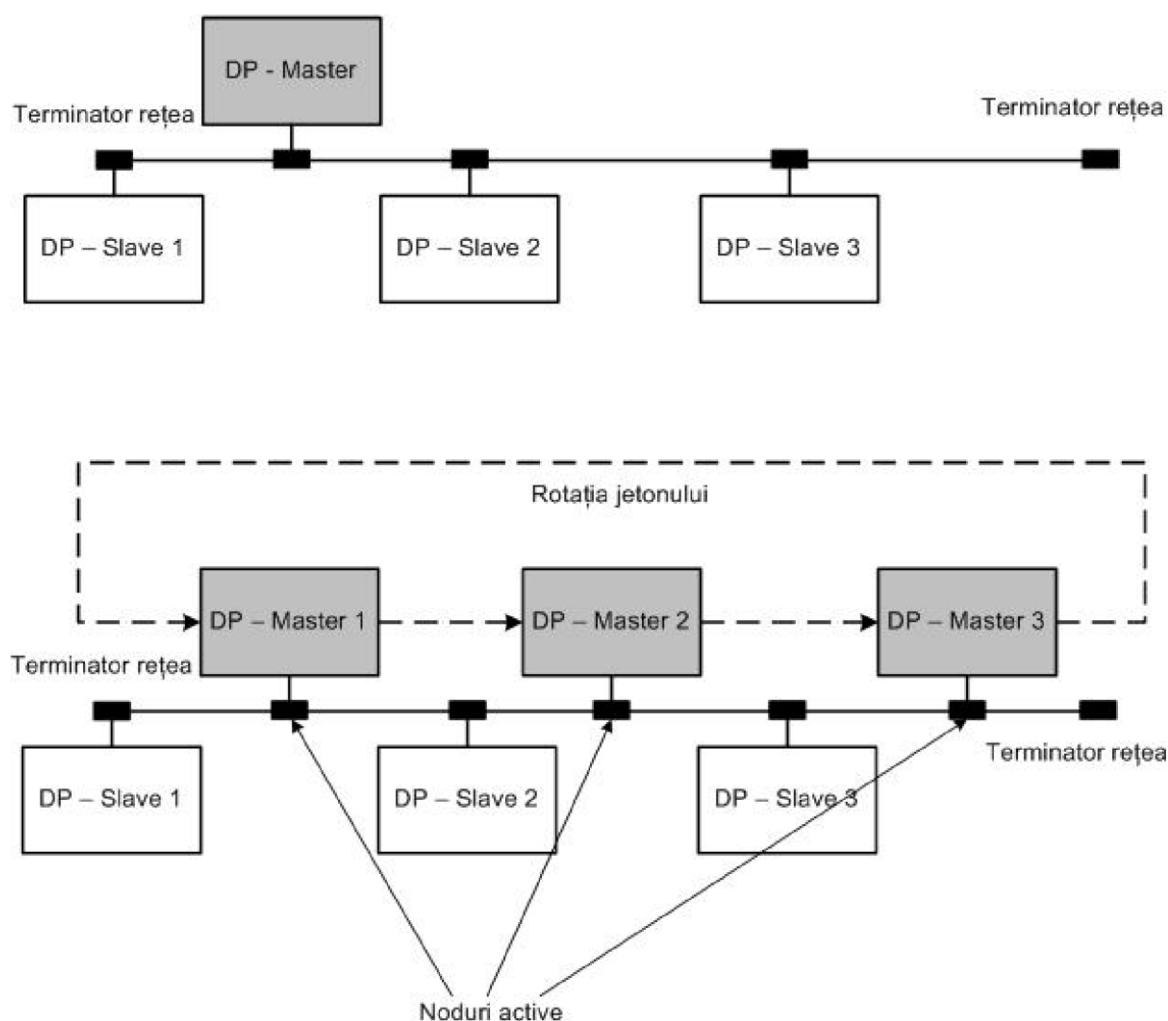


Fig. 5.2. – Structura unei rețele Profibus

Pentru configurarea unei rețele PROFIBUS este necesar, în afara realizării fizice, de un dispozitiv de configurare, care poate fi un calculator dedicat cu facilități de comunicare pe rețea. Alternativa la utilizarea acestor dispozitive este utilizarea unui PC, căruia i se atașează o interfață specială de comunicație pe PROFIBUS. Pe aceste calculatoare rulează programe speciale care dispun de o bibliotecă specială de descriptori de dispozitive (GSD). Această bibliotecă este livrată de constructorul de dispozitiv dar poate fi completată și de utilizatori atunci când apar noi dispozitive.

Configurarea unui master de Profibus DP înseamnă:

- configurarea adreselor dispozitivelor slave;
- stabilirea tipului dispozitivelor slave;
- stabilirea altor parametri pe care îi cere dispozitivul slave.

După înscrierea configurației în master, acesta realizează următoarele acțiuni:

- stabilește rețeaua și inițializează DP slave;
- scrie parametri și configurația în DP slave;
- citește informația de diagnostic de la DP slave pentru a verifica dacă aceștia au acceptat parametri și configurația;

- începe schimbul de date cu dispozitivul DP slave.

Unul din scopurile organizării rețelelor de automate este acela de a realiza coordonarea acțiunilor mai multor automate care conduc o automatizare complexă. Cea mai simplă cale de a realiza acest lucru este utilizarea unui vector de stare care este actualizat pe rețea de către fiecare dispozitiv și care este folosit în programe pentru coordonarea acțiunilor.